

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	In progress	11
Bug	Operator-blocked	1
Bug	Queued	42
Bug	Ready for testing	4
Task	In progress	12
Task	Queued	30
Task	Ready for testing	2
TOTAL		102

Items

ATM ID	Type	Status	Severity	Description
BOB-008	Bug	Operator-blocked	—	OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): COMPLETE THE CAPTCHA FLOW ONCE Session establishment path is DECIDED: the operator drives the interactive CAPTCHA flow at /api/v1/auth/rutracker/captcha then /login once, and the proxy stores the resulting bb_session. The copy paste path and the cookies-file autoload path were both offered and NOT chosen. THE BLOCK NARROWS BUT DOES NOT LIFT: the operator must physically drive the flow — no agent can solve a CAPTCHA (§11.4.52 honest operator_attended boundary). AGENT PREPARED BEFORE THE HAND-OFF, so the opera

ATM ID	Type	Status	Severity	Description
				single attempt succeeds rather than discover broken endpoint mid-flow: verify both endpoints are reachable and correctly wired end-to-end, verify the session is actually PERSISTED after successful post (not merely accepted), and verify a stored session is actually USED by a subsequent search. Honest limitation to state up front: a stored bb_session expires, so this path requires repeating — that cost was accepted with the decision. This answer is recorded as consumed DATA per §11.4.35 — it is the operator's state choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applies to decisions). — prior item text follows — RuTracker automated login blocked by CAPTCHA
BOB-065	Task	Queued	High	Lava P2: Egress diagnosis and VPN-host SOCKS routing (containers pkg/egress)
BOB-066	Task	In progress	High	Lava P3: BOBA_UPSTREAM_PROXY in download proxy + qBitTorrent-go + Jackett + compose environment forward
BOB-068	Bug	In progress	High	RD2-00: unattributed, unreviewed Auto-commit mechanism pushing to main
BOB-069	Task	In progress	High	RD2-40: §11.4.238 QA-discovery-channel ledger ongoing coverage-escape backfill
BOB-074	Task	In progress	Medium	RD2-07: DDoS-class testing fully absent from mandated test-type matrix
				OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): UNKNOWN — INSTRUMENT THE NEXT OCCURRENCE The operator does not currently know which host produces the +0500 Auto-commit fast-forward so the three candidate answers (second Clause session / intentional rsync job / stale job) all remain open. DECIDED ACTION: stop hunting host this session cannot reach (§11.4.6 — remote state is not knowable without access) and instrument the mechanism so the next occurrence identifies itself. Forensic capture
BOB-077	Task	Queued	High	

ATM ID	Type	Status	Severity	Description
				add: committer identity, hostname, timezone offset, push timing, and the git remote used, recorded at Auto-commit time into a tracked forensic log. ACCEPTANCE: the next Auto-commit event yields a captured record naming its origin host — at which point this item resolves to one of the three original branches with evidence rather than a guess. This is the §11.4.101 reversible-move: it costs little, blocks nothing, and converts a recurring mystery into a self-identifying event. This answer is recorded as consumer DATA per §11.4.35 — it is the operator’s stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5)) bounded-verdict discipline applied to decisions — prior item text follows — RD2-10: Identify second host running the Auto-commit rsync/ssh mechanism (OPERATOR-DECISION) RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit push script OR retire it RD2-13: Retroactive Fable-xhigh code review the substantive Auto-commit diffs RD2-15: Create BOB-064..067 workable items the four Lava-porting findings (closes GA-05) RD2-18: Create top-level Boba proxy/merge-service v1.0.0 readiness ledger (closes GA-10) RD2-21: Complete/verify README Tracked-Items + Status Documents table row-completeness (GA-07 remainder) RD2-25: HelixQA Challenge entry exercising all three start.sh subcommands end-to-end against real compose stack RD2-28: Live compose bring-up + verify rutra ReDoS regex bounds deployed to container (closes runtime half of GA-12)
BOB-078	Task	Queued	High	
BOB-080	Task	Queued	High	
BOB-082	Task	Queued	High	
BOB-085	Task	Queued	Medium	
BOB-088	Task	Queued	Medium	
BOB-090	Task	In progress	Medium	
BOB-093	Task	In progress	High	
BOB-094	Task	In progress	Medium	

ATM ID	Type	Status	Severity	Description
				RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with §11.4.122 fault injection
BOB-095	Task	In progress	Medium	RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos.py for side triangle
BOB-097	Task	Queued	Low	RD2-32: Author DDoS-class coverage for export/download-proxy/merge endpoints (canonical impl of RD2-07)
BOB-100	Task	Queued	Low	RD2-39: Bump submodules/jackett one comm (canonical impl of RD2-09)
				OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): DEFER PAST v1.0 Go --profile go parity remains a goal but is LONGER a v1.0.0 release blocker. RW-10..13 n to a post-1.0 milestone and stop gating the tag Nothing advertised is removed, so §11.4.122 s clean and no §11.4.90 Obsolete closure is warranted. The BOB-141 measured fact stand unchanged: the Go container's Dockerfile run ONE binary binding only :7187; nothing bind :7186 or :7188 despite compose setting PROXY_PORT/BRIDGE_PORT. CLAUDE.md's po map already describes what the container ACTUALLY does and needs no amendment un this decision. This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inferen and supersedes any prior agent-chosen defau on this question. Options not chosen are nam above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item follows — GA-19/RW-09: Is --profile go parity a release goal? (gates RW-10..13) — OPERATO DECISION
BOB-101	Task	Queued	High	
				OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): KEEP 0.0.0.0 + MANDATORY AUTH GUARD The tunnel STA LAN-reachable — no bind address changes to 127.0.0.1. The operator explicitly chose to
BOB-102	Task	In progress	Medium	

ATM ID	Type	Status	Severity	Description
				preserve access from other devices on the network. THE DECISION CARRIES A BINDING OBLIGATION: a permanent §11.4.135 regression guard MUST land that FAILS THE BUILD if an LAN-reachable route ever stops demanding authentication. The operator accepted 0.0.0.0 THE CONDITION that guard exists — without this decision is unprotected and the item is N closeable. Guard requirements: enumerate routes from the authoritative source (FastAPI app, Go handlers, boba-jackett) never a hand maintained list; resolve real auth coverage n grep for the string ‘auth’ (§11.4.201 real-condition); deliberately-public routes exempt ONLY via a checked-in list with per-entry justification, enumerated as honest gaps never silent; golden-TRUE + golden-FALSE fixtures p §11.4.107(10). This answer is recorded as consumer DATA per §11.4.35 — it is the operator’s stated choice, not an agent inference and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item follows — RW-05: LAN-exposure threat mode bind tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION
BOB-104	Task	In progress	Medium	§11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge
BOB-106	Task	Queued	Medium	§11.4.238 followup: §11.4.84 quiescence-check helper for the unattributed auto-commit path
BOB-107	Task	Queued	Medium	§11.4.238 followup: pre-dispatch existence check for subagent task-brief source inputs
BOB-109	Task	Ready for testing	Medium	BOB-074 followup: scaling-class test coverage absent from mandated test-type matrix
BOB-110	Task	Queued	Medium	BOB-074 followup: UX-class test coverage (accessibility/usability) absent
BOB-111	Task	In progress	High	BOB-074 followup: configure real rate limiting for boba’s 3 public HTTP endpoints
BOB-114	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				BOB-074 followup: self-validation golden-bad fixture for the rate-limit detector
BOB-121	Task	Ready for testing	Important	External watchdog for the forced-logout architectural gap (task #85, incident #3)
BOB-135	Bug	Ready for testing	Low	Test isolation: test_list_hooks_after_create fails bulk suite (Permission denied /config)
BOB-137	Bug	In progress	High	Merge service on 7187 wedges while the same process still serves 7186 (GIL starvation by our spinning thread)
BOB-141	Task	Queued	Low	CLAUDE.md claims the Go profile serves 7186/7187/7188 but its container binds only 7186 — doc contradicts the Dockerfile
BOB-143	Bug	Queued	Medium	Orphaned .worktrees/ dirs (46M, unresolvable gitdir) pollute gate scan scope and manufacture false BOB-126-class findings
BOB-149	Bug	Queued	Low	Managed-plugin count diverges 43/42/48 across constitution, CLAUDE.md and the README badge; the badge is hand-maintained and unguarded
BOB-150	Task	Queued	Low	pre_build_verification.sh invariant labels read 44 but only 35 invariants are labelled
BOB-151	Task	Queued	Medium	CM-SCRIPT-DOCS-SYNC is a named gate with no implementation, and 24 of 36 scripts have no companion doc
BOB-152	Bug	Queued	Medium	Constitution sweep walks vendored third-party code: 82% of one gate's 38,291 findings come from submodules/
BOB-154	Bug	Ready for testing	Medium	Host venv and production container run different starlette versions (1.4.1 vs 1.6.0)
BOB-156	Bug	Queued	Medium	BOB-145 event-loop regression guard is load-sensitive and flaky: 8786ms under host load vs 900-1500ms ceiling calibrated on a quiet host
BOB-159	Bug	Queued	Medium	OPERATOR DECISION (2026-08-26, §11.4.66): REPAIR, THEN RE-VERIFY UNTIL STABLE TESTS WARM ./start.sh path keeps running the ownership repair against a live stack, but warms, re-verifies, and repeats until a pass finds nothing new. Options not chosen: refuse the repair while the stack is up and direct the operator to

ATM ID	Type	Status	Severity	Description
				–recreate; stop the stack on the warm path to (correct but ends ‘warm’); accept and document the window. WHY THIS MATTERS MORE THAN NORMAL ITEM: this is the exact defect the walk-002-user-owned-downloads feature exists to fix. The –recreate path was already fixed to order precondition -> down -> repair -> up so the walk sees a tree no container can write. The warm path was not, so a container can create a new non-owned file AFTER the repair has passed the directory — the ownership problem leaking in through the other door. REQUIRED BY THE CHOSEN OPTION, and it is the hard part: the MUST be BOUNDED, and on giving up it MUST report honestly that ownership is UNPROVEN rather than silently exiting 0 (§11.4.201(6) — loop that stops finding new files because it ran out of iterations returns the same quiet zero genuinely clean tree). An active download creating files faster than the walk completes the realistic non-converging case and must be named in the give-up message. Recorded as consumer DATA per §11.4.35 — the operator’s stated choice, not an agent inference. Options chosen are named so a future reader does not litigate a settled call. — prior item text follows Reported-Via: §11.4.202 reporting directive 1 on 2026-08-21T19:01:17Z Reported-By: T041 independent review (IMPORTANT-2), partially remediated What (the report, verbatim): The –recreate path was fixed this round: run_ownership_precondition -> stack_down -> run_ownership_repair -> stack_up, so the repair walks a tree no container can write to. That ordering is now asserted behaviourally by the new checks in tests/unit/test_start_reload_recreate.sh, each killed by its paired 1.1 mutation (11/11 RED-OK). The WALK path is NOT covered by that fix and this item tracks the remainder. On a warm ./start.sh the gate runs before THIS invocation brings anything up, but if the stack is ALREADY running, containers write while the repair walks. A

ATM ID	Type	Status	Severity	Description
				container can then create a new non-operator owned file BEHIND the walk, after which the completion marker records complete over a t that is not, and those stragglers are never repaired without a manual –force. BOUNDED dismissed: after any successful pass the marker is valid for the scope fingerprint and the repair short-circuits without walking (marker_is_val so the window requires a stale-or-absent marker AND a live stack together. Declaring a new scope entry re-arms the marker, which is exactly how that combination arises in practice. NOT DONE THIS ROUND, with the reason stated rather than hidden: the clean guard is a cheap marker-only probe mode on scripts/ownership_repair.sh that answers has-this-scope-been-repaired without walking the tree. The existing –dry-run cannot serve: it deliberately SKIPS the marker fast-path (FORCE=0 && DRY_RUN=0 guard) and always walks, so using it as a warm-start probe would walk the tree twice on every start. Adding the mode requires editing ownership_repair.sh, which another stream was actively editing during this round; duplicating marker_is_val into start.sh instead would be a near-identical fork (11.4.251). Deferred to this item rather than done unilaterally or silently. The misleading comment at the warm-start call site (which claimed the gate runs before any container writes) has been corrected to state this bound honestly. Affected scope / file-scope manifest start.sh (warm-start dispatch, run_ownership_gate call site); scripts/ownership_repair.sh (marker fast-path) Reproduction / context: 1. Ensure the stack UP. 2. Change config/owned_paths.yaml so the scope fingerprint changes (this re-arms the marker by design, data-model E2). 3. Run a warm ./start.sh (no –recreate). 4. The ownership repair walks and chowns the declared tree while containers are still running and able to write it. Acceptance criteria: A warm ./start.sh can complete an ownership repair while a container

ATM ID	Type	Status	Severity	Description
				that writes to a declared location is running. Either the repair is deferred with an actionable refusal naming <code>–recreate</code>, or the stack is quiet for the walk. Asserted behaviourally in <code>tests/test_start_reload_recreate.sh</code> alongside the existing <code>PRECONDITION_BEFORE_DOWN / REPAIR_AFTER_DOWN / REPAIR_BEFORE_UP</code> checks, each with a paired 1.1 mutation that it. Reported-Via: §11.4.202 reporting directive + on 2026-08-21T19:02:48Z Reported-By: AI W (the report, verbatim): Independent §11.4.2 review (IMPORTANT-3) found two of the feat strongest automated checks were never executed by any runner: (1) <code>tests/ownership/test_container_writes_owned_files.py</code> — the §11.4.115 RED-turned-regression-guard for FR-011/FR-007, proven via a <code>docker-compose</code> revert mutation — was moved out of <code>tests/integration/</code> (commit 58d340a, to escape an autouse fixture hang) but no runner (<code>ci.sh</code>, <code>test.sh</code>, <code>run-all-tests.sh</code>) was ever extended to cover <code>tests/ownership/</code>. (2) <code>tests/pre_build/test_.sh</code>, the §1.1 paired-mutation meta-tests for the scripts/<code>pre_build/check_cm_.sh</code> gate family was executed by NOTHING — self-reported honestly in commit 04742d7’s own message (‘STILL OPEN (reported, not fixed): <code>tests/pre_build/</code> is executed by NOTHING’) but never tracked as a workable item (a §11.4.197 loss-of requirements gap) and never wired into script <code>pre_build_verification.sh</code> invariant 30, which globbed only <code>tests/unit/test_.sh</code>. Affected scope file-scope manifest: <code>ci.sh</code>; <code>scripts/pre_build_verification.sh</code> invariant 30 (CM-BA. UNIT-TESTS-EXECUTED) Reproduction / context: <code>grep -rn test_container_writes_owned_files ci.sh test.sh run-all-tests.sh scripts/</code> returned zero runner h (only docs/specs mentioned the path); <code>grep</code> in <code>scripts/pre_build_verification.sh</code> showed invar 30’s for-loop globbing only <code>tests/unit/test_.sh</code>,
BOB-160	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				never tests/pre_build/test_.sh. Acceptance criteria: ci.sh gains a runtime-gated pytest test ownership/ stage (skips honestly, rc=0, when n podman/docker present); scripts/pre_build_verification.sh invariant 30's glob co both tests/unit/test_.sh and tests/pre_build/test_*.sh, verified by an extracted standalone of the invariant's exact block reporting a non zero RAN count that includes files from both directories. Reported-Via: §11.4.202 reporting directive t on 2026-08-21T19:31:12Z Reported-By: BOB-0 remediation, residual finding What (the report verbatim): §11.4.69 names CM-NO-FAIL-OPE SKIP as one of three mandatory pre-build gat it audits sink-side probe helpers and FAILs if code path converts an empty or unreachable response into a PASS-counting SKIP for a feat class with a sink-side probe. This project does have it. Why this matters now rather than in abstract: the BOB-092 remediation just removed TWO fail-opens of exactly this class from test e2e/test_live_stack_evidence.py — the nnmcl SKIP-on-404 (already removed at 7baef2b) and surviving sibling at test_iporrents_is_authenticated_in_search th skipped on 'not authenticated and status != success' while asserting 'treating as transient outage'. That assertion was false for rejected credentials (upstream_http_403) and for a broken container (plugin_env_missing), both which are definitive product failures the pro already classifies via error_type. Two fail-ope of one class, found one at a time, is the §11.4. extend-to-all-cases signal and the §11.4.238 coverage-escape signal together: the regime c not surface either, an agent reading code did. remediation's own guards are AST-structural lethal under three discriminating mutations, they LIVE IN THE FILE THEY GUARD, so dele that file evades them entirely. A pre-build gat the durable home because it audits the corpu
BOB-161	Task	Queued	High	

ATM ID	Type	Status	Severity	Description
				rather than one file. Honest boundary (§11.4.201(1)). This item does NOT claim the two remediated guards are unguarded — they are guarded, with runtime evidence (13 passed in 97.36s against live stack). It claims the CLASS has no corpus-wide detector, so the next instance in a different file is invisible again. Affected scope / file-scope manifest: scripts/pre_build/ (gate absent); tests/e2e/test_live_stack_evidence.py (guards currently live inside the file they guard) Reproduction context: grep -rl 'CM-NO-FAIL-OPEN-SKIP' scripts/ tests/ returns exactly ONE hit, and it is tests/e2e/test_live_stack_evidence.py — the file the guards live in, not a gate. Control needle: same query for CM-OWNERSHIP-INVARIANT (no gate that does exist) returns 3 files, so the instrument can see gate tokens and the single is a real absence, not a blind zero (§11.4.201(1)(b)). Acceptance criteria: A scripts/pre_build gate named CM-NO-FAIL-OPEN-SKIP exists, is wired into scripts/pre_build_verification.sh, and audits sink-side probe helpers for code paths converting an empty/unreachable/error response into a PASS-counting SKIP for a feature class HAS a sink-side probe. It ships a golden-TRUE fixture (a real fail-open -> gate FIRES) and a golden-FALSE-with-carrier (an honest topology geo skip, and a comment merely MENTIONING the phrase -> gate MUST NOT fire), per §11.4.107(10)/§11.4.201(1). Paired §1.1 mutation makes the gate FAIL before the gate is trusted. Reported-Via: §11.4.202 reporting directive + on 2026-08-21T19:35:02Z Reported-By: BOB-107 BOB-107 remediation, residual What (the report, verbatim): The TESTS for both guards are now executed: pre-build invariant 30's glob was extended from tests/unit + tests/pre_build also cover tests/hooks. MEASURED before/after the glob expanded to 27 suites with 0 under tests/hooks while 3 existed there; it now expands to 27 with 3 under tests/hooks. All three pass. But the GUARDS themselves are still invoked by nothing.
BOB-162	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				check-brief-inputs.sh is honestly conductor-r rather than automatic: a brief's required-inp list lives in free-form prose that the Agent too structured input does not expose, so an automatic prompt-scraping gate would itself an unproven pattern-match. Its seam is the dispatch procedure, and that is a documenta + habit change, not a hook. unattributed-com guard.sh is different and is why this item exists. Run against real history it names 14 violating commits since the last tag (and 21 bare Auto-commit subjects exist across all refs). Wiring into the pre-build or commit seam AS-IS would therefore refuse every commit immediately, pre-existing debt nobody can fix in the moment. That is precisely the 11.4.224(E) brownfield-adoption shape, and 11.4.66/11.4.122 put the choice with the operator, not with an agent inventing a ratchet. The options, stated so the decision is a decision and not a default: (a) immediate hard floor – the seam refuses until 14 are attributed; (b) one-time monotone-decrease ratchet (the 11.4.135 pattern) – snap 14 as the baseline, the count may fall and may never rise, so day one is green and the disease cannot spread; (c) changed-commits-only – go only commits created from now on, with a scheduled deadline for the historical 14; (d) report-only for a stated period, then escalate. Recommendation, with the reasoning rather than just the pick: (b). It is the pattern this constituent already uses for exactly this situation, it makes the debt visible and bounded immediately, and cannot be satisfied by deleting the guard's name (11.4.227 closes that gaming channel). But it is the operator's call and this item is not closed until that answer is recorded. Honest boundary (11.4.6): this item does NOT claim the 14 commits are harmful in themselves – it claims their CONTRIBUTION is absent and that nothing currently prevents the 15th. Affected scope / file-scope manifest: scripts/hooks/unattributed-commit-guard.sh; scripts/hooks/check-brief-

ATM ID	Type	Status	Severity	Description
BOB-163	Bug	In progress	Medium	inputs.sh; scripts/pre_build_verification.sh; scripts/commit-push-all.sh Reproduction / context: Both guards run correctly by hand and are covered by passing tests (9/9 and 15/15, e proven non-bluff against a no-op stub). Neither invoked by scripts/pre_build_verification.sh or scripts/commit-push-all.sh, so neither exerts standing detection pressure (11.4.226: a guard with no execution seam is not coverage; registration is not coverage). Acceptance criteria: Each guard is invoked by a named seam, OR carries a registered deferral pointing to this item. For unattributed-commit-guard.sh specifically, the operator has answered the adoption question below and the answer is recorded as consumer DATA – never an inverted ratchet. OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): YES — 429 IS RESPONSIVE IF Retry-After IS WELL-FORMED A 429 carrying a VALID Retry-After header counts as the sibling endpoint being RESPONSIVE. Assertion (c) of the DDoS challenge accepts 2xx OR a well-formed 429 (positive integer seconds or valid HTTP-date — PARSE not merely present/non-empty). DEGRADED remains: connection failure, timeout, any 5xx and a 429 with absent or malformed Retry-After. RESIDUAL RISK, accepted knowingly and to be stated in the script source per §11.4.6: a genuinely wedged endpoint that happens to answer 429-with-Retry-After would pass — the Retry-After parse narrows that window, it does not close it. Options (b) limiter-aware drain and (c) exempt healthz probe were both offered and NOT chosen. STILL OPEN, not covered by this decision: the detector counts 429 only, not 503 though the script header says ‘429 (or equivalent)’; counting 503 would collide with the crash detector’s 5xx tally. Decide when BOB-1 lands limiters on :7185 and :7189. This answer recorded as consumer DATA per §11.4.35 — i

ATM ID	Type	Status	Severity	Description
				the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions — prior item text follows — Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:41:08Z Reported-By: BOB-114 remediation, pre-existing defect surfaced by BOB-111 landing a real limiter What (the report verbatim): PRE-EXISTING, NOT INTRODUCED and proven so rather than asserted: git diff shows assertion (c) is BYTE-IDENTICAL to HEAD and the failure reproduces against the unmodified HEAD script. What changed is not the challenge but the SYSTEM: BOB-111 appears at least partially delivered, because :7187 now returns real 429s where it previously returned none. :7185 and :7189 still produce zero 429s. This is a 11.4.248-class corrosion risk and that's why it is filed rather than left as a footnote: run_all_challenges.sh will now fail INTERMITTENTLY, and an intermittent red triggers everyone to re-run until green, at which point real regression is dismissed as 'probably the flaky rate-limit one'. THE DECISION, which is operator call under 11.4.66 and which I deliberately did NOT invent: Does a 429 from WORKING limiter count as the sibling endpoint being RESPONSIVE? (a) YES – a 429 proves the endpoint is alive and correctly protecting itself. Assertion (c) accepts 2xx OR 429, and only a connection failure / 5xx / timeout counts as degraded. Risk: a genuinely wedged endpoint that happens to answer 429 would pass. (b) No, keep requiring 2xx, but make the challenge limiter-aware: drain or wait out the window before the sibling probe (retry-after is served if the budget is knowable rather than guessed). Probe siblings on a path the limiter exempts (healthz-class route), so the isolation question is asked without spending the bucket.

ATM ID	Type	Status	Severity	Description
				Recommendation with reasoning, not just a p (a) composed with a bounded liveness check 429 carrying a well-formed Retry-After IS evidence of a live, correctly-behaving service and (b) makes the challenge slower and coup it to a window value that will drift. But the semantics of ‘responsive’ here are a product judgement, so the answer is recorded, not assumed. RELATED, stated as fact rather than folded in: the detector counts 429 only, not 50 though the script header says ‘429 (or equivalent)’. Counting 503 would collide with crash detector’s 5xx tally. Worth deciding wh BOB-111 lands limiters on :7185 and :7189. Affected scope / file-scope manifest: challen scripts/ddos_resilience_challenge.sh assertio cross-endpoint isolation; run_all_challenges.s which invokes it Reproduction / context: Assertion (c) requires a sibling-endpoint prob answer ^2 (a 2xx). :7187 now enforces a real limiter (measured live: x-ratelimit-limit: 120, ratelimit-remaining: 86, retry-after: 45, serve uvicorn). A full challenge run sends roughly . requests to :7187, so sibling probes fired duri the OTHER endpoints’ tiers land on an exhaust bucket and receive 429 – which assertion (c) scores as ‘endpoint degraded’. Timing- dependent: one run measured PASS=5 FAIL=5 the UNMODIFIED HEAD script against the sam live stack ~30s later measured PASS=7 FAIL=0 SKIP=2. Acceptance criteria: The operator h answered the classification question below, t answer is recorded as consumer DATA, and assertion (c) implements it. The challenge the produces the same verdict across 3 consecuti runs against a rate-limited stack (11.4.50 deterministic consistency), and the fix ships a paired 1.1 mutation proving assertion (c) still catches a genuinely degraded sibling endpoint narrowing it must not blind it (11.4.201(1)). Reported-Via: §11.4.202 reporting directive on 2026-08-21T19:56:53Z Reported-By: BOB-1
BOB-164	Bug	In progress	Medium	

ATM ID	Type	Status	Severity	Description
				UX-class coverage, discovered by the new axe-core suite on its first live run What (the report verbatim): This is a REAL user-facing defect, not a test-tuning artifact, and it was found by the automated regime rather than by a human squinting at the page – which is exactly the 11.4.238 posture the project is aiming for. The static-grep oracle would NOT have found it. Measured: the served root ships an empty pre-hydration, so any check reading the raw HTML audits a page nobody sees. The violation only exists in the hydrated DOM, which is why 11.4.170 requires a rendered oracle and forbids value-equality assertions as the proof a UI is correct. Severity reasoning, stated rather than assumed: this is user-visible and affects the primary dashboard heading, but it degrades legibility rather than breaking function, and the surface is operator-facing rather than public. Medium, not High. The failing test was left FAILING on purpose (11.4.238) instead of silenced or marked xfail. tests/ux/ currently reports 1 failed, 16 passed; that 1 is this defect. Anyone reading a red UX suite should read it as this item, not as flakiness – and when this is fixed the suite goes fully green, which is the signal it is closed. HONEST SCOPE LIMIT (11.4.6): on the dashboard landing view was scanned. The jaxet/* sub-routes and the ng-serve-hosted :4200 route set were NOT audited – the commands to close both are recorded in docs/testing/ux_accessibility.md. So this item's 21 nodes are a floor, not a total. Affected scope file-scope manifest: the Angular dashboard served at http://localhost:7187/ (same compile SPA as frontend/); production component CSS not test files Reproduction / context: Run tests/ux/test_live_dashboard_accessibility.py against the running merge service. axe-core v4.13.0, scanning a real Playwright-rendered JS-hydrated DOM, reports color-contrast violations on 21 nodes. Measured pairs: .brand / h1 text #9d00 on background #3c3f41 = 1.43-1.62:1 (WCAG A

ATM ID	Type	Status	Severity	Description
BOB-165	Bug	Queued	High	large-text floor is 3:1); tagline #808080 on #3c = 2.68:1 (body-text floor is 4.5:1). Acceptance criteria: axe-core reports ZERO color-contrast violations against the live rendered dashboard with the fix made in production component (rather than by relaxing the assertion or excluding the rule (11.4.120: reconcile to the correct mechanism, never weaken the check). The existing tests/ux/ suite is the guard and already fails today, so the RED is captured – closure requires it flipping GREEN against the live surface, which is runtime-class evidence 11.4.226. Reported-Via: §11.4.202 reporting directive on 2026-08-21T20:00:08Z Reported-By: surface while capturing closure evidence for BOB-09 diagnosed rather than worked around What report, verbatim): This is a STALE-ABI-AFTER-INTERPRETER-UPGRADE defect, not a missing package. The package is installed; its compiler half is built for the previous interpreter. That distinction matters because ‘pip install rpds-py’ style advice can appear to succeed while leaving the 313 artefact in place. WHY IT WAS NOT NOTICED EARLIER, stated as fact: the paths that DO work all avoid system python3. ci.sh selects an interpreter through _select_python; the long running suites in this session ran .venv/bin/python -m pytest and passed. So the automated regime is green while the DOCUMENTED operator command is broken – an 11.4.238-shaped gap, since the escape was found by an agent capturing evidence rather than by the regime. WORKAROUND (measured, not theorised): PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 with the needed plugins passed explicitly (-p pytest_timeout ...) avoids the schemathesis autoload that drags in jsonschema -> referenced -> rpds. That is a workaround, NOT the fix: it silences the import path rather than repairing the install, and it will surprise the next person

ATM ID	Type	Status	Severity	Description
				who runs the documented command verbatim. This is a RELATED, and worth deciding together rather than twice: BOB-154 wants .venv rebuilt on 3.12.13 to match the container (which runs CPython 3.12.13 while both system and venv run 3.14.0). There are therefore THREE interpreters in play. Whoever rebuilds the venv should confirm rpython resolves for the TARGET interpreter afterwards, or this same class reappears one directory over the HONEST BOUNDARY (11.4.6): this item does NOT claim any test is wrong or any product code is broken. The product and the venv-run suites are unaffected. What is broken is the documented entry point. Affected scope / file-scope manifest: host user site-packages (~/.local/lib/python3/site-packages/rpds/); every CLAUDE.py documented ‘python3 -m pytest ...’ invocation, any tooling that uses system python3 rather than .venv/bin/python. Reproduction / content: python3 -m pytest tests/e2e/test_live_stack_evidence.py -q --import-mode=importlib -> ModuleNotFoundError: No module named ‘rpds.rpds’, raised during collection via the schemathesis plugin autoloader -> jsonschema -> referencing._core -> rpds. MEASURED root cause: system python3 is 3.13 but ~/.local/lib/python3/site-packages/rpds/ships/rpds.cpython-313-x86_64-linux-gnu.so – an extension built for 3.13. rpds/init.py does ‘from .rpds import *’, and a 313-tagged .so is not importable by 3.14, so the submodule genuinely does not exist for this interpreter. The venv is CORRECT and unaffected: .venv/lib64/python3.14/site-packages/rpds/ships/rpds.cpython-314-x86_64-linux-gnu.so and .venv/bin/python -c ‘import rpds’ succeeds. Acceptance criteria: python3 -m pytest tests/unit/-v --import-mode=importlib – the command CLAUDE.md documents – reaches collection without ModuleNotFoundError, OR CLAUDE.md is corrected to document the supported runner explicitly. Either way the documented command and the working command agree (11.4.99: a

ATM ID	Type	Status	Severity	Description
				guide that misleads is the documentation-lay equivalent of a PASS-bluff). WORKAROUND S EFFECTS MEASURED 2026-08-21 — the recipe not free, and the item previously implied it w PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 disables ALL plugin autoload, not just the bro one, so anything the suite silently relied on m be re-enabled by hand: - It BREAKS 5 pre-exis env-dependent tests in tests/unit/ test_plugin_rutacker.py — TestConfig::test_default_mirrors, test_env_mirrors_override, test_get_env_with_default, test_get_mirrors_from_env_empty, test_get_mirrors_from_env_whitespace — wh need an autoloaded env plugin. PROVEN not caused by any in-flight change: running HEA OWN copy of that file under identical flags produces the same 5 failures. - It makes -timeout=60 (set in pyproject.toml) an UNKNOWN ARGUMENT unless -p pytest_time is passed explicitly, so pytest.mark.timeout is silently inert under the naive recipe — a test believed to be time-bounded is not. - With autoload ON and only schemathesis disabled no:schemathesis), that same file is 96/96 gree CONSEQUENCE FOR ANYONE USING THE WORKAROUND: -p no:schemathesis alone is strictly better than blanket autoload-disablin where it suffices, because it removes only the broken plugin. Where the blanket form IS us -p pytest_timeout must be added or timeouts inert, and the 5 env-dependent failures must recognised as workaround artifacts rather th filed as defects. That last point is the real risk workaround manufactures failures that look exactly like product defects. This does not ch the root cause (the venv's rpds ships a cpython-313 ABI tag CPython 3.14 cannot imp or the fix (rebuild the venv — BOB-154). It documents that the interim recipe has a blas

ATM ID	Type	Status	Severity	Description
				radius, so nobody reads a workaround artifact as a regression.
				WHAT. download-proxy exposes two Server-Sent Events routes. They are the same expensive class — long-lived connections that hold a worker as a generator for their lifetime — but only one rate-limit classed: routes.py:801 @router.get('/search/stream/{search_id}') @_rl('sse_stream') classed routes.py:150 @router.get('/theme/stream') async def stream_theme(...) <- NO limiter decorator _rl(cls) resolves to limiter.limit(limit_for(cls)), so /search/stream bound to the sse_stream class while /theme/stream falls through to the application default. Measured live on the running stack: /api/v1/theme/stream reports x-ratelimit-limit 120. PROVENANCE + A CORRECTION WORTH KEEPING (§11.4.6). This was surfaced by the BOB-109 scaling agent, whose report framed 'sse_stream_limit_decorator is defined and imported/applied nowhere ... SSE falls through the 120/minute default: 24x less protected'. The framing is WRONG and was NOT filed as given. The agent searched for one symbol NAME; the wiring uses a different mechanism (@_rl('sse_stream')), and it IS applied — to /search/stream. Verified by reading routes.py:795-806 and the _rl helper at routes.py:46-51, with a control needle confirming other *_limit_decorator symbols show real use in api/init.py so the zero-hit was not a blind search. The agent's MEASUREMENT (120 on /theme/stream) was correct and is what makes this a real finding; its MECHANISM was not. If halves are recorded so the next reader does not re-derive the same wrong cause. WHY IT MATTERS. An unclassed SSE endpoint is the cheapest way to pin server resources: each connection is held open, and the default class permits 120/min of them. The declared sse_stream class exists precisely because this route shape needs a tighter bound than ordinary
BOB-167	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
BOB-168	Task	In progress	Low	GETs. ACCEPTANCE. (a) A decision, recorded, whether /theme/stream belongs in the sse_stream class or genuinely warrants the default — this is a policy question, not automatically a bug to patch. (b) If it belongs in sse_stream, the decorator is applied and a test drives BOTH S routes and asserts each returns its INTENDED class limit from x-ratelimit-limit, so a future route added without a class is caught. (c) A group that enumerates SSE-shaped routes and fails any that carries no explicit rate-limit class — in general form, so the third SSE route does not repeat this. (d) Honest boundary: this does not claim 120/min is exploitable in this deployment with network_mode host and no reverse proxy; every caller shares the 127.0.0.1 bucket, which BOB-111 measured and recorded separately. CLAIMED. No change made. The limit values were read from headers, never driven to exhaustion — the limiter is per-IP and shared with concurrent agents on this host (§11.4.11). WHAT. scripts/run_all_challenges.sh:66 lists “scaling_horizontal_challenge.sh” in its challenge set. challenges/scripts/scaling_horizontal_challenge.sh does not exist. sed -n ‘66p’ scripts/run_all_challenges.sh “scaling_horizontal_challenge.sh” \$ ls challenges/scripts/scaling_horizontal_challenge.sh ls: cannot access ...: No such file or directory VERIFIED independently, not taken on report — surfaced by the BOB-109 scaling agent and re-checked here by direct invocation. WHY IT MATTERS. Whether this is cosmetic or a §11.4.201 gate-honesty defect depends entirely on how the runner treats a missing entry, and that is the thing to determine: if it SKIPS silently, the challenge bank advertises coverage it does not have (a §11.4.266 claim-vs-reality row with no passing challenge behind it); if it FAILs, the runner is permanently red for a reason unrelated to the system under test, which traps readers to ignore it. Neither outcome is

ATM ID	Type	Status	Severity	Description
				acceptable; they need different fixes. ACCEPTANCE. (a) Determine and record the runner's actual behaviour on the missing entry by invoking it, not by reading it. (b) EITHER author the challenge, OR remove the entry — with §11.4.124 discipline: check git history for whether it once existed and was deleted, since a silently-dropped challenge is the more interesting defect. (c) If the runner silently skips missing entries, that is its own finding: a missing challenge must be loud (§11.4.3 SKIP-with-reason at minimum), never absent-and-quiet. SEVERITY: Low as a defect, but it sits on the challenge-coverage seam, so (c) may deserve its own item. CORRECTION 2026-08-22 (§11.4.6) — THIS ITEM'S PREMISE WAS FALSE, and the error was minor. Recorded openly rather than quietly edited. scaling_horizontal_challenge.sh EXISTS. It is a submodule/challenges/challenges/scripts/scaling_horizontal_challenge.sh, executable, 1024 bytes, dated Aug 7. So do all 14 other entries in the meta-runner. It was never deleted: added to the roster at 1c959ba, the challenge itself added in the submodule at 873c0b1, and a pickaxe search across both repositories and all branches finds ZERO deletions. HOW I GOT IT WRONG: run_all_challenges.sh reads submodule/challenges/challenges/scripts/. I checked challenges/scripts/ — a DIFFERENT directory belonging to a DIFFERENT, glob-based runner. Two similarly-named directories; I measured the wrong one. That is a §11.4.201(9) field-identity error. Worse, I wrote that it was “verified by invocation”: I had listed a directory and never run the aggregator, so the claim overstated the evidence class as well as getting the answer wrong. THE LESSON, which generalises past this item. In the SAME commit I correctly caught a sibling agent making this exact class of error BOB-167 — a zero-hit produced by searching a symbol name — and applied a control needle there. Then I omitted the needle one paragraph later on my own measurement. The distinction

ATM ID	Type	Status	Severity	Description
				that would have caught it: A CONTROL NEED PROVES THE INSTRUMENT CAN SEE; IT DOES NOT PROVE IT IS POINTED AT THE THING UNDER TEST. §11.4.201(7)(b) as commonly applied guards blindness; it does not guard naming. The needle for THIS query would have been to resolve the path the runner itself reached not to confirm that some directory listing was WHAT IS ACTUALLY WRONG, and it is worse than what I filed. The missing-entry path runner challenges by definition, so it can be exercised safely. Run against a checkout where the submodule is absent — the ordinary state of clone made without –recursive — the REAL runner (byte-identity sha-asserted, not a repl reports: PASS: 0 FAIL: 0 SKIP: 16 TOTAL: 16 OBSERVED EXIT CODE: 0 An ENTIRELY UN-R BANK REPORTS SUCCESS. Not one dangling error tolerated — the whole suite silently passes without executing anything. That is §11.4.201 false-null verbatim, and it is observed-reachable rather than constructed (§11.4.115(G)). REVIS ACCEPTANCE. (a) ANSWERED, though not as filed: the runner reports a loud, counted SKIP and then never blocks, because the exit expression reads only the FAIL count. (b) The dangling-entry half is VOID — nothing is dangling. (c) The real work is the exit contract roster entry the runner cannot execute must be indistinguishable from a healthy run. See decision recorded in docs/qa/BOB-168/decision_missing_vs_skip.md. RELATED, REPORTED NOT FIXED: scripts/pre_build_verification.sh:1792 carries the same SKIP/MISSING conflation one level up, for the other runner. Filed separately. WHAT. The §11.4.65 markdown-export mand requires every in-scope doc to ship .html/.pdf twins. 286 of the 326 .html files under docs/ (excluding dist/ and node_modules/) are panc FRAGMENTS — they begin at
BOB-169	Bug	In progress	Medium	
BOB-170	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				WHAT. The BOB-109 scaling suite gates its two timing tests on <code>loadavg_1m/nproc <= 0.75</code> — on this 8-cpu host, a 1-minute load average at or below 6.0 — and SKIPS loudly otherwise. The gate is correct and was adopted for a good reason (§11.4.201(8)): the author validated a 2.10 growth-exponent threshold, then observed it FALSE-FAIL correct code at load 11.06 (baseline span 2.136), and at load 15-23 measured baseline spans of 2.358 against injected-cubic mutants 2.278-2.331 — the signal is smaller than the noise and the two are not separable in either direction. Gating was the honest response. THE PROBLEM. The gate has consequently NEVER RUN under its own precondition. Every recorded run today measured load 9.15-23; the independent reviewer measured 9.66 during review; the conductor measured 6.45 at its lowest. GREEN polarity WAS observed — at loads 9.4-11.6 (span 1.882) and in three stability runs (1.759/1.832/1.848) — but never once beneath the shipped ceiling. The author states this honestly as an inference from data rather than a run the author can paste, and the reviewer confirmed the inference is sound (quiescence is strictly more favourable). Sound inference is still not an observed run. WHY THAT MATTERS (§11.4.222). A registered, topology-present guard that never executes is indistinguishable from a guard that would fail if it did. Nothing currently ensures it ever runs: no freshness contract, no scheduled quiet-window invocation, no tracked item — which is precisely the unexecuted-standing-guard class §11.4.226 names, where registration gets treated as coverage while execution is nobody's job. The forensic precedent in that anchor is two standing guards that, when finally executed, immediately emitted latent FAILs. ACCEPTANCE. (a) Execute both quiescence-gate timing tests on a host whose 1-minute loadavg genuinely $\leq 0.75/\text{cpu}$ — no agent fleet, no concurrent build — and capture the run as an artifact showing the resolved load reading

ATM ID	Type	Status	Severity	Description
				alongside the measured span exponent, so the precondition is provable from the artifact and not asserted. (b) Record the observed GREEN polarity in the item and in the test source, replacing the current honest-inference note. the gate FAILS under genuine quiescence, that the finding this item exists to surface, and it reopens the threshold-calibration question rather than being worked around. (d) Establish freshness contract per §11.4.226: how stale a quiescent verdict may be before the gate could as uncovered again, so this does not silently return to never-executing. HONEST BOUNDARY This item does not claim the gate is wrong. The evidence points the other way — quiescence is strictly more favourable than the loads where GREEN was already observed. It claims only that the run has not happened, and that an unexecuted guard cannot be cited as coverage. PROVENANCE. Raised as IMPORTANT-2 by the independent Fable-substrate reviewer of the BOB-109 work, and independently corroborated: the reviewer verified the SKIP is loud and precise, its resolved numbers, and confirmed the ship tests do SKIP at today's load. WHAT. Both rate limiters key their per-client bucket on the LEFTMOST element of X-Forwarded-For when TRUST_FORWARDED_FOR is enabled: download-proxy/src/api/rate_limit.py:117-123 (:7187) and the new std limiter in plugins/download_proxy.py (:7186) which was deliberately built to exact policy parity with it. That element is CLIENT-SUPPLIED An honest reverse proxy APPENDS the peer address to the RIGHT of whatever arrived, so the leftmost entry is whatever the client sent — attacker-controlled by construction, not by misconfiguration. Consequences with the flag (1) rotating a forged leftmost value mints an unlimited sequence of fresh buckets, which is a total bypass of the limit the flag is supposed to make MORE accurate; (2) it also defeats the L
BOB-171	Bug	Queued	Low	

ATM ID	Type	Status	Severity	Description
				idle-reap cap, since each forged identity is a distinct key — the bucket map is a memory-growth surface bounded only by the cap, and cap's eviction then discards the budgets of RE clients to make room for forged ones. NOT EXPLOITABLE AS DEPLOYED TODAY, and that's why this is Low, not High. TRUST_FORWARDED_FOR defaults OFF at both sites, and the deployed stack runs network_minimal host with no reverse proxy in front (verified across all five compose services), so peer addresses are real client IPs and no NAT collision occurs. The defect is latent: it arms the moment someone puts a proxy in front and turns the on to recover real client IPs — which is exactly the situation the flag exists for. The failure mode is therefore 'correct-looking configuration change silently disables the limiter', not 'currently broken'. PROVENANCE. Raised as MINOR-3 by the independent reviewer of the BOB-111 rate-limiter work and independently confirmed by the implementing agent, which noted it at BOTH source sites as a tracked follow-up rather than fixing it in that change. Filed because neither agent has write access to the tracker. WHY IT COVERS BOTH PORTS. :7186' limiter was built to deliberate policy parity with :7187's, including this parsing. Fixing one and the other would leave the two surfaces disagreeing on client identity — worse than the shared defect, because it becomes surface-dependent and untestable as a single invariant. ACCEPTANCE. (a) Replace leftmost-element parsing with a trust-aware resolution at BOTH sites: either rightmost-minus-N-trusted-hops, trusted-proxy CIDR allowlist where only a peer inside the allowlist may assert XFF at all — the choice is a deployment-topology decision and should be recorded, not assumed. (b) A test that forges a rotating leftmost element and asserts the bucket key does NOT rotate, per site. (c) Paired §1.1 mutation: restore leftmost parsing and the test must FAIL. (d) Negative control

ATM ID	Type	Status	Severity	Description
				(§11.4.201(1)): with TRUST_FORWARDED_FORWARDING OFF, the header must be ignored entirely and keying must fall back to the peer address — a guard that starts honouring XFF when the flag is off would be a worse defect than the one being fixed. (e) Honest boundary: this closes header forgery bypass; it does not make per-IP limiting fair behind NAT, where many real users legitimately share one address. NOT CLAIMED. No change made. Both sites still parse leftmost flag still defaults off.
BOB-172	Bug	Ready for testing	High	WHAT. The rutracker SEARCH endpoint is refused by bot protection while the site itself is fully reachable. Measured unauthenticated 2026-08-21 (no cookie file read, no credential value in any artifact — §11.4.10): GET /forum/index.php -> 200, 96283 B, WHAT. download-proxy/src/api/hooks.py:96-100: def _save_hooks(hooks: list[dict[str, Any]]) -> None: try: os.makedirs(os.path.dirname(HOOKS_FILE), exist_ok=True) with open(HOOKS_FILE, 'w') as f: json.dump(hooks, f, indent=2) except Exception as e: logger.error(f'Failed to save hooks: {e}') catches EVERY exception, logs, and returns None. The signature returns None, so the caller has no channel to learn the write failed. Both call sites then report success unconditionally: :152 _save_hooks(hooks):174 _save_hooks(hooks):175 logger.info('Created hook: ...'):175 logger.info('Deleted hook: ...'):156 return HookResponse(hook_id=..., ...):176 return {'message': 'Hook deleted', ...} USER-VISIBLE CONSEQUENCE, which is why this is High and not a code-hygiene nit. A user POSTs a webhook, receives HTTP 200 and a hook_id, and the hook was never written — their automation silently never fires, and the API told them it exists. Symmetrically, a user DELETES a hook, is told 'Hook deleted', the file still holds it, and it fires again after the next restart. In both directions the product reports the opposite of what happens.
BOB-173	Bug	Ready for testing	High	WHAT. download-proxy/src/api/hooks.py:96-100: def _save_hooks(hooks: list[dict[str, Any]]) -> None: try: os.makedirs(os.path.dirname(HOOKS_FILE), exist_ok=True) with open(HOOKS_FILE, 'w') as f: json.dump(hooks, f, indent=2) except Exception as e: logger.error(f'Failed to save hooks: {e}') catches EVERY exception, logs, and returns None. The signature returns None, so the caller has no channel to learn the write failed. Both call sites then report success unconditionally: :152 _save_hooks(hooks):174 _save_hooks(hooks):175 logger.info('Created hook: ...'):175 logger.info('Deleted hook: ...'):156 return HookResponse(hook_id=..., ...):176 return {'message': 'Hook deleted', ...} USER-VISIBLE CONSEQUENCE, which is why this is High and not a code-hygiene nit. A user POSTs a webhook, receives HTTP 200 and a hook_id, and the hook was never written — their automation silently never fires, and the API told them it exists. Symmetrically, a user DELETES a hook, is told 'Hook deleted', the file still holds it, and it fires again after the next restart. In both directions the product reports the opposite of what happens.

ATM ID	Type	Status	Severity	Description
				and the only trace is a log line nobody is watching. THIS IS THE §11.4.252 SHAPE. The path combines two dangerous capabilities from that anchor's taxonomy — MUTATION of a shared resource (a filesystem write) and EXTERNAL SIDE EFFECT (hooks are outbound calls the system will or will not make) — so it required to FAIL CLOSED: verify the precondition, refuse when it cannot be satisfied, and surface the refusal. Instead it fails open, a bare 'except Exception:' that only logs is the exact anti-pattern §11.4.252 enumerates. At the product layer it is also a §11.4.201(6) false-nu successful write and a swallowed failure are indistinguishable to the caller. PROVENANCE Surfaced as an out-of-scope observation by the independent reviewer of the BOB-135 test-isolation work — this swallow is what turned that defect into an 'assert 0 == 1' mystery, because the EACCES on /config was logged and discarded while the endpoint kept returning. Verified here directly from source before filing (the function body and both call sites read above), not taken on report. Recorded as a §11.4.238 discovery-channel escape: found by agent reading code during an unrelated investigation, not by the automated QA regime; the coverage gap is a defect of equal standing to the defect itself. ACCEPTANCE. (a) _save_hook propagates failure — raise, or return a status callers must consume. (b) Both endpoints translate a persistence failure into an HTTP error (500-class), never a success body; a create that did not persist must not return a hook_id. (c) test drives each endpoint with the hooks file unwritable (read-only dir or a patched open raising OSError) and asserts a non-2xx status AND that a subsequent GET does not list the phantom hook — assert on the user-observable outcome, not on the log line. (d) Paired §1.1 mutation: restore the swallow; the test must FAIL. (e) Audit the same file for sibling swallow — this is a pattern, and one instance is rarely

ATM ID	Type	Status	Severity	Description
				alone. (f) Honest boundary: this does not claim the write currently fails in production; it claims that WHEN it fails the user is told the opposite and the BOB-135 investigation shows it does fail in at least one real environment. NOT CLAIMED. No change made. No assessment of how often write fails in the operator's deployment. RECORDING A SHELL ERROR OF MY OWN (§11.4.6): the first version of this description was written with the anti-pattern snippet inside backticks in a double-quoted shell argument, the shell ran it as command substitution and the text was replaced by nothing — the stored description read 'and is the exact anti-pattern'. This is the SECOND time this session that backticks-inside-double-quotes has corrupted content (the first mangled a commit message). Fixed here by editing through a Python client with no shell quoting in the path. Noted because a silently-truncated defect description is exactly the kind of quiet corruption §11.4.201(7)(c) warns about — the path is part of the instrument, and it failed without erroring. WHAT. A corrupt hooks file is silently indistinguishable from "no hooks configured" and the next create then DESTROYS every existing hook while returning HTTP 200. Three defects on one path, filed together because fixing any one alone leaves the data loss reachable. — download-proxy/src/api/hooks.py:104-112. <code>_load_hooks</code> wraps the read in <code>except Exception: logger.error(...)</code> and fails through to return <code>[]</code>. A truncated or malformed <code>hooks.json</code> is therefore reported to every caller as an empty, healthy hook list. Confirmed at source. A2 — the consequence, and the reason this is High. <code>create_hook</code> loads, appends, saves. Given a corrupt file that load turns into <code>[]</code>, the save writes a one-element list over the top. Every previously-configured hook is gone. Measure the implementing agent against the ALREADY-FIXED tree, so this survives the BOB-173 write
BOB-174	Bug	In progress	High	

ATM ID	Type	Status	Severity	Description
				failure fix: BEFORE on disk : ['prod-hook-0', 'prod-hook-1', 'prod-hook-2'] GET reports : 200 {'hooks': [], 'count': 0} <- claims ZERO hooks configured DELETE reports : 404 {'detail': 'Hook not found'} <- for a hook that IS in the file PO reports : 200 hook_id=3c8a5930-... AFTER on disk : ['3c8a5930-...'] Three prod hooks destroyed HTTP 200 throughout, nothing surfaced to the user. A5 — _save_hooks writes with a plain open(path, "w"). A crash, ENOSPC, or a kill m write truncates the file in place. That is precisely the corruption A1 then reads as “no hooks” and A2 overwrites. The same codebase already has the correct pattern: theme_state.py:115-126 use tmp-file + os.replace. WHY THE THREE ARE CORRUPT ITEM. A5 manufactures the corrupt file, A1 misreads it as empty, A2 destroys the content. Fixing only A1 leaves truncation reachable; fixing only A5 leaves an existing corrupt file as a data-loss trigger; fixing only A2 leaves the API lying about what is configured. The chain is the defect. DELIBERATELY NOT FIXED UNDER BOB-173, and the reasoning is sound. The implementing agent identified all three while auditing sibling swallows as that item required and declined to fix them in the same change because: they change GET semantics on an endpoint the frontend consumes (200 -> 5xx) they need a CORRUPT-file reproduction rather than the UNWRITABLE-file one BOB-173 built and they need a design decision that is genuinely not obvious — a MISSING hooks file legitimately means “no hooks configured”, while a CORRUPT one does not, and today’s code cannot tell them apart. Expanding BOB-173’s scope to cover this would have meant shipping that decision unexamined. ACCEPTANCE. (a) Distinguish MISSING from CORRUPT: a missing file remains an empty list; a corrupt one is an error, never silently []. (b) Decide and RECORD what GET on corruption — 5xx, or 200 with an explicit degraded marker the frontend can render. This is the design decision, and it should be stated

ATM ID	Type	Status	Severity	Description
				rather than inferred from whatever the patch happens to do. (c) create/delete MUST NOT overwrite a file they could not parse — refuse not clobber (§11.4.252: mutation plus external side effect must fail closed). (d) Make the write atomic via tmp + os.replace, reusing theme_state.py's existing pattern rather than inventing it (§11.4.28). (e) Tests asserting the USER-OBSERVABLE outcome, not log lines: see corrupt file, assert GET does not claim zero hooks, assert a create refuses rather than destroying, and assert the file still holds the original hooks afterwards. (f) Paired §1.1 mutation per guard. (g) NEGATIVE CONTROL (§11.4.201(1)): a MISSING file must still yield an empty list and a working create; a VALID file must behave exactly as today. A fix that makes every load fail closed by failing always is not a fix. A3, RECORDED SEPARATELY, NOT PART OF THIS CHAIN. VALID_EVENTS and HookEvent are two sources of truth for one closed set. Verified identical today, unguarded against divergence if they diverge a hook registers successfully and then silently never fires. One assertion pinning them would close it. NOT CLAIMED. No change made. The BOB-173 write-failure fix is real and orthogonal — it makes a FAILED write honest; it does nothing about a SUCCESSFUL write of wrong data derived from a misread file. WHAT. The workable-items update seam guards the status-location invariant in one direction only. After BOB-166, update --status <terminal> on an Issues-located row is refused. The mirror is still open: update --location Fixed --status <non-terminal> exits 0 and creates a row that update's own validator immediately rejects. Verified empirically by the independent review of BOB-166: update --location Fixed --status 'In progress' -> exit 0 validate -> FAILS, naming the row (check (f), fixedLocationNonTerminalStatus). So the command can still mint a state its own validate call refuses. That is the §11.4.196(F)
BOB-175	Task	Queued	Low	

ATM ID	Type	Status	Severity	Description
				shape at the seam layer: the invariant is CONFIGURED in validate but not ENFORCED every write path that can violate it. WHY IT V DELIBERATELY LEFT OPEN IN BOB-166, and that was right. Three reasons, all checked rather than asserted: (1) the real corpus has ZERO instances of this direction against TEN of the direction BOB-166 closed — the asymmetry in data matched the asymmetry in the code; (2) detective coverage ALREADY existed and demonstrably fires, so unlike BOB-166's direction this cannot rot silently; (3) and the load-bearing one — reopenCmd documents and SANCTIONS transient Fixed-plus-non-terminal state via its location Fixed operator override. A naive preventive guard at the update seam would collide with that override's semantics. Closing this therefore requires a design decision about how the two interact, which is a different question from the one BOB-166 was scoped to answer, and expanding that item would have meant shipping the decision unexamined. ACCEPTANCE. (a) Decide and RECORD how a preventive guard on this direction coexists with reopenCmd's sanctioned override — this is the actual work, and the answer should be written down rather than inferred from whatever the patch does. Candidates worth weighing: extend the reopen path explicitly, require an override flag on update mirroring reopen's, or leave the direction detective-only with the rationale recorded so the asymmetry is a decision rather than an oversight. (b) If a guard lands, it reuses terminalStatuses() — BOB-166 established a predicate source specifically so the two directions cannot drift. (c) Paired §1.1 mutation (d) NEGATIVE CONTROL (§11.4.201(1)), and it's the sharp one here: the sanctioned reopen path MUST still work. A guard that breaks reopenCmd's documented override is a false-positive refusal, and worse than the gap it closes because it breaks a workflow operators are told to use. HONEST BOUNDARY. This is not a live

ATM ID	Type	Status	Severity	Description
				data defect. Zero corpus instances, and the detective gate catches it at the next validate. filed so a scoped, deliberate omission stays visible as a tracked decision instead of living in a code comment where the next reader will mistake it for an oversight (§11.4.197: a started thread reaches a documented terminal state, never quiet abandonment). PROVENANCE. Raised by the implementing agent of BOB-160, a known asymmetry it deliberately did not check independently verified and endorsed as an acceptable scope boundary by that item's reviewer, on the condition that it become a tracked item rather than a comment. This is the item. WHAT. <code>_get_enabled_trackers()</code> in <code>download-proxy/src/merge_service/search.py</code> gates <code>rutracker</code> on <code>RUTRACKER_USERNAME</code> and <code>RUTRACKER_PASSWORD</code> only. It never consults <code>RUTRACKER_COOKIES</code>. So an operator configured with cookies alone — no username, no password — never has <code>rutracker</code> enabled at all, and the tracker is silently absent from even fan-out rather than failing visibly. WHY THAT IS INCOHERENT WITH THE REST OF THE CODE. <code>_search_rutracker</code> treats <code>COOKIES</code> as the PREFERRED auth path, not a fallback. And the <code>nnmclub</code> sibling DOES check its own <code>*_COOKIES</code> variable. So the same codebase holds three positions at once: cookies preferred at the search site, cookies ignored at the enablement gate, cookies honoured for a sibling tracker. WHY THAT MATTERS MORE THAN IT LOOKS. CLAUDE.M. documents a standing operator mandate (2026-08-15) that per-tracker Netscape cookie files at <code>\${TRACKER_COOKIE_DIR}/cookies_<tracker>.txt</code> are auto-loaded into <code>.env[<TRACKER>_COOKIES]</code> before every <code>boba-svc</code> restart, install Stage 6 and <code>start.sh</code> boot — and <code>rutracker</code> is named in that set. So the SANCTIONED, documented configuration path for this tracker produces exactly the state this
BOB-176	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				gate ignores. An operator who follows the documented instructions gets a tracker that never runs, with no error to explain it. FAILURE MODE. Silent absence, not a visible failure — same §11.4.201(6) false-null shape as BOB-172 one layer earlier. BOB-172 fixes a tracker that RAN and was REFUSED being reported as empty this is a tracker that never ran at all, and the merged result simply has one fewer contribution with nothing to indicate it. PROVENANCE. For by the BOB-172 implementing agent while tracing the enablement path to locate the starting handling defect. Recorded here rather than fix in that change: it is a separate defect on a separate seam with its own oracle, and folding in would have widened BOB-172 past its captured evidence. ACCEPTANCE. (a) The enablement gate honours RUTRACKER_COOK as an independently sufficient credential, matching what _search_rutracker already prefers and what the nnmclub sibling already does. (b) A test asserting that a cookies-only configuration ENABLES rutracker, driving the real _get_enabled_trackers rather than a replacement. (c) Paired §1.1 mutation restoring the username password-only gate — the test must go red. (d) NEGATIVE CONTROL (§11.4.201(1)): a configuration with NO credentials at all must leave rutracker DISABLED. A fix that enables unauthenticated tracker is worse than the gate because it produces failing searches instead of absent ones. (e) Audit the other trackers' gate the same asymmetry rather than fixing only one that was noticed — three positions in one codebase suggests nobody has checked them together (§11.4.118). HONEST BOUNDARY. Not verified against a live cookies-only deployment read from source by the BOB-172 agent and recorded on its report. The reading is specific and checkable, but whoever takes this should confirm by invocation before relying on it. BOB-177 Bug Queued High WHAT: BOB-172 wired an identical 4-line HTTP-refus

ATM ID	Type	Status	Severity	Description
				guard at five private-tracker fetch sites in download-proxy/src/merge_service/search.py (rutracker cookie :1390, rutracker credential :1468, kinozal :1648, nnmclub :1761, iptorrent :1934). Only the rutracker COOKIE path is exercised by tests. EVIDENCE (reviewer-auth mutation R1, per 11.4.194(6)(d), during the BOB-172 independent review): deleting ONLY kinozal guard wiring (search.py:1655-1658) while leaving the classifier intact left the full merge_service suite at 883 passed, ZERO failure. The suite cannot see four of the five sites. FAILING SCENARIO: a refactor drops or subtly breaks the wiring at kinozal, nnmclub, iptorrents, or rutracker-credential. A 403 at the site silently folds back to status=empty with error=None -- the exact BOB-172 signature -- no test reddens. FIX DIRECTION (reviewer preferred): collapse the five duplicated wiring into one shared helper, e.g. <pre>_check_search_response(tracker_name, status, body) -> bool, so there is ONE copy to test and</pre> sixth site cannot be added unguarded (11.4.2 byte-identical-fork extraction). Alternative: parametrise the guard tests across all five sites with per-site stub sessions. ACCEPTANCE: mutating the wiring at ANY of the five sites reddens at least one test. Prove it by running same R1 deletion at each site in turn. BOB Bug Queued Medium WHAT: download-proxy/src/merge_service/search.py:1638-1640 handles a failed kinozal LOGIN response with login_resp.status not in (200, 301, 302): logger.error(...); return [] -- and sets NO diagnostic. FAILING SCENARIO: Cloudflare returns 403 on takelogin.php. The kinozal checker reads status=empty, error=None. That is the BOB-172 signature exactly: a refusal reported to the user as an empty result set. CONTRAST establishing this is an oversight, not a design choice: the rutracker and nnmclub login failures DO set diagnostics (upstream_captcha / auth_failure), and the iptorrents login failure

ATM ID	Type	Status	Severity	Description
				falls through to the search fetch where BOB-172 new guard catches it. Kinozal is the one leg w neither. FIX DIRECTION: stash _classify_upstream_http_status(login_resp.status) before the early return, or an auth_failure diag for the status-in-(200,301,302)-but-no-co case. ACCEPTANCE: a stubbed 403 on the kino login endpoint produces a non-None error on kinozal chip, with a RED captured against the pre-fix code first (11.4.115). BOB-179 Task Queued Medium WHAT: the BOB-172 independent review demonstrated two further paths that still report a refusing or unreachable tracker as an empty result. Both are PRE-EXISTING and neither is a regression from BOB-172, but the fix evidence log presents the five-site coverage without stating the boundaries (11.4.194(5) requires un-analysed dimensions explicit gaps, never silently assumed safe). G. -- exception before resp.status is read. Review probe B, captured: a connection-refused rutracker yields status=empty, error=None, http_status=None, metadata.errors=[], metadata.status=completed. Each _search_* method swallows exceptions (except Exception logger.error; return results) before _search_or error handling can see them, so a tracker that DOWN is indistinguishable from one that is genuinely empty. GAP B -- soft refusal at HTTP 200. Reviewer probe A, captured: _classify_upstream_http_status(200, <cf-chl body>) returns None, which is CORRECT by design (a 2xx is a usable response; body-marker triggering would risk the over-fire the negative controls exist to prevent). But all five search use aiohttp's default allow_redirects=True, so session-expiry 302 -> login-page-200 chain path to zero rows and reports empty. FIX DIRECTION Gap A needs the exception to reach a diagnosis rather than being swallowed. Gap B needs a DISTINCT detector (final-URL check or login-f marker) with its own RED -- explicitly NOT a widening of the status-code trigger.

ATM ID	Type	Status	Severity	Description
				ACCEPTANCE: both gaps closed with their own REDs, or explicitly closed per 11.4.112 with evidence. Immediate sub-task: append a state gaps paragraph to docs/qa/BOB-172/fix_evidence_20260822.log. BOB-180 Bug Queued Low WHAT: download-proxy/src/routes.py:727-745 sets status=captcha_required with a hardcoded message naming RuTracker 'RuTracker requires CAPTCHA. Use /api/v1/auth/rutracker/captcha'. FAILING SCENARIO: a whois search returns zero results and the captcha-flavoured diagnostic came from NNMClub's Turnstile, not RuTracker. The user is told to visit a RuTracker captcha endpoint for an NNMClub problem. SEVERITY BOUNDED: the full error and tracker_stats travel in the same response payload, so the truth is present and a client that reads them is not misled -- only the headline is wrong. The branch was revived by BOB-172's error propagation (correctly: suppressing that propagation would recreate the same false-negativity one layer up, 11.4.247) and is already covered in the routes layer by tests/unit/api_layer/test_routes_coverage.py:782. FIX DIRECTION: derive the message from the tracker(s) that actually erred rather than hardcoding one. SCOPE NOTE: api/ was owned by a sibling structure during BOB-172; the author was correct not to touch it. ACCEPTANCE: an NNMClub-only captcha diagnostic on a zero-result search produces a headline naming NNMClub. BOB-181 Bug Queued High WHAT: scripts/generate_markdown_exports.sh accepts no arguments. It unconditionally scans PROJECT_ROOT/*.md, docs/ recursively and scripts/ recursively (scan loops at lines 105/111/116 as of 2026-08-23; they were 96-100 when this was filed — line numbers shifted with the BOB-169 fix, the substance is unchanged and was independently verified by execution: invoking the generator with an explicit path produced an out-of-scope export and NOT the requested one), and PROJECT_ROOT is derived

ATM ID	Type	Status	Severity	Description
				from BASH_SOURCE (line 17). A caller who w 'bash scripts/generate_markdown_exports.sh docs/qa/SOME/file.md' gets a WHOLE-TREE r with the argument silently discarded. WHY I MATTERS: the caller reasonably believes the was scoped to one file. It was not. In a shared checkout with parallel work streams this regenerates any export whose .md is newer t its derived files anywhere in the tree -- landin artifacts in other streams' scopes and, with th BOB-068 auto-commit hazard, into another stream's commit. CLASS: a false affordance -- script's interface implies a capability it does n have (11.4.201: an interface must mean what claims). FIX DIRECTION: either accept option path arguments and honour them (scoping th run to exactly those files), or REFUSE with a c message when arguments are passed. Silently discarding them is the one option that must r remain. ACCEPTANCE: passing a path either scopes the run to it, or exits non-zero naming unsupported argument. A RED capturing tod silent-discard behaviour first (11.4.115). LIVE INSTANCE, MEASURED 2026-08-23 (this is no longer hypothetical): During the BOB-169 fix conductor invoked the generator with an exp path to regenerate ONE document. The argum was discarded and the whole tree was scanne generated docs/qa/BOB-174/DESIGN_DECISIO {html,pdf,docx} at 11:50 -- a SIBLING STREAM document, while that stream's author was sti editing the source (.md mtime 12:00). The exp were therefore stale on arrival, and addition carried the pre-fix BOB-169 charset defect. Th sibling author independently reported them stale in its own hand-off, having no idea anot stream had created them. SECOND-ORDER LESSON worth keeping with this item: the conductor's attempt to verify the blast radius ALSO failed, and failed silently. 'git status -- porcelain \ grep -E "\.(html\ pdf)\$"' returned only its own files and read as clean – but doc BOB-174/ is an UNTRACKED DIRECTORY, whi

ATM ID	Type	Status	Severity	Description
BOB-182	Task	In progress	Medium	porcelain collapses to a single line, so the gre was structurally incapable of seeing the files inside it (11.4.201(7)(a): matched the wrong surface; the quiet zero read as absence). The instrument that DOES see it is 'find OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): KEEP THE MONOTONE RATCHET §11.4.224(E) brownfi adoption answer, recorded as consumer DAT. the adoption model for CM-EXPORT-CHARSE VALID is the MONOTONE-DECREASE RATCHE The count may only go down, never up. Immediate hard floor, changed-code-only-wit deadline, and per-corpus phase-in were all offered and NOT chosen. This closes half of BOB-182 — the half that was genuinely the operator's. THE REMAINING HALF IS A REAL DEFECT STILL OWED: BASELINE is a hardcod constant, so the ratchet does not actually ratch — on improvement the gate PASSES and PRIM the value to lower it to, but nothing lowers it, the gate keeps permitting regression back to original count after the corpus heals. The fix must NOT collapse §11.4.249 role separation: gate that writes its own threshold during a p build run becomes a PRODUCER as well as a GATE. Acceptable shapes include a persisted baseline the gate READS but never writes wit lowering via a separate explicitly-invoked command, or a gate that FAILS loudly when t live count is below baseline so drift becomes actionable refusal. This answer is recorded a consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inferen and supersedes any prior agent-chosen defau on this question. Options not chosen are nam above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item follows — WHAT: CM-EXPORT-CHARSET-VAL (pre-build invariant 50) adopts its 301 pre-existing violations via a monotone-decrease

ATM ID	Type	Status	Severity	Description
				ratchet rather than a hard floor. Two things are allowed. (1) THE ADOPTION DECISION IS THE OPERATOR'S, NOT THE SCRIPT'S. 11.4.224(E) requires the brownfield adoption question be SURFACED to the operator and the answer recorded as consumer DATA – never an inverse ratchet. What exists today is a header comment plus a BOBA_EXPORT_CHARSET_BASELINE override: that DOCUMENTS a decision the agent made, it does not RECORD an answer the operator gave. The independent reviewer judged the choice defensible and would not reverse a hard floor on 301 violations makes the build unreachable, which 11.4.234 forbids; 11.4.103 favours the reversible option) but correctly holds that it is not yet compliant. Options for the operator: immediate hard floor (BASELINE=0) keep the monotone ratchet / per-corpus phase in / changed-code-only with a scheduled full-corpus deadline. (2) THE RATCHET DOES NOT ACTUALLY RATCHET. BASELINE is a hardcoded constant. On improvement the gate PASSES and PRINTS the value to lower it to, but nothing lowers it, so the gate keeps permitting regressions all the way back to 301 after the corpus heals. The header now says tightening is MANUAL rather than claiming automatic behaviour that does not exist (11.4.6), but the honest statement is a stopgap, not the fix. WHY IT WAS NOT BUILT WITH THE FIX: a gate that writes its own threshold during a pre-build run becomes a PRODUCER as well as a GATE (11.4.249 role separation), and that is a design change the operator should approve rather than receive as a side effect. ACCEPTANCE: the operator's adoption answer recorded as consumer DATA; and either persisted baseline the gate lowers and never raises (with the role-separation question unanswered), or an explicit decision that manual tightening is acceptable.
BOB-184	Task	Queued	Medium	Three icon-glyph controls (.bridge-retry, .thermometer, .toggle, .caret) render text made of non-BMP code points.

ATM ID	Type	Status	Severity	Description
				points. axe-core declines to decide their contrast, reporting them under the incomplete channel. It fails with messageKey nonBmp and resolving NEITHER fgColor NOR bgColor, so the Python script recomputation in docs/qa/BOB-164/axe_contrast_scan.py cannot decide either: 40 such nodes across 16 palette x mode combinations. BOB-164 round 2 stopped them from being scored as silent passes by naming them GLYPH_UNMEASURED, printing them on every run and fencing them, so any NEW unmeasured glyph node blocks. What remains genuinely unproven is their contrast itself: as non-text user interface components they owe 3:1 under WCAG SC 1.4.11, and no oracle in this repo measures that today. Acceptance: a measurement path that resolves the effective foreground and background for glyph controls (for example reading getComputedStyle colour against the composed backdrop, or replacing the glyphs with inline-block carrying explicit fill tokens), each of the three controls shown to clear 3:1 in all 16 palette x mode combinations, and the GLYPH_UNMEASURED fence shrunk by exactly the nodes then proven. docs/qa/BOB-164/axe_contrast_scan.py scans the static dist with no backend, so every node whose existence depends on fetched data is absent from the page it measures: the results table renders empty, so .type-badge and .quality-badge never appear, and the hooks list renders empty, so .status pills never appear. That is precisely why the BOB-164 round-1 regression went unseen in a rendered DOM even though it was scanned in all 16 themes, and it is why round-2 had to close the class in the arithmetic oracle instead. Those nodes are now covered by frontend/src/app/models/style-contrast.spec.ts which extracts declared foreground-on-fill pairs from the stylesheets, but that is an ARITHMETIC claim about declared colour pairs, not a rendered-pixel one: it cannot see opacity applied
BOB-185	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
BOB-187	Bug	In progress	—	by an ancestor, a composited backdrop, or a cascade this repo’s static extractor does not model. Acceptance: the scan drives the dashboard with seeded fixture data (a stub backend, a route fixture, or an injected component harness) so the badge and status nodes are present in the DOM, axe measures them directly, and a control needle proves the rendered oracle sees a seeded sub-floor badge before any clean result from it is believed. scripts/ownership_precondition.sh consumes SAME unreviewed .env-driven scope as ownership_repair.sh and creates probe files inside it, but it is NOT fenced. T028 remediation added ownership_path_fence() to scripts/lib/ownership.sh and wired it into ownership_repair.sh, closing the path-escape class there; the precondition was outside the agents file scope and still accepts whatever config/owned_paths.yaml expands to. Blast radius is lower than a recursive chown (it wrote probe files rather than mutating ownership of existing tree) but the INPUT is identical and equally unreviewed, so the same QBITTORRENT_DATA_DIR value that would have driven a filesystem-wide chown will drive probe file creation at an arbitrary location. Note the escape is empirically confirmed for the repair path, not merely reasoned: during T028 remediation a probe declaring the verbatim shipped entry with QBITTORRENT_DATA_DIR hung the suite, and ps showed the real artifact executing “find / (! -uid 1000 -o ! -gid 1000) -printf ...” before it was killed by verified pid. Acceptance: ownership_precondition.sh calls SAME ownership_path_fence() predicate from scripts/lib/ownership.sh (never a second dial 11.4.251), refuses fail-closed on a rejected entry and ships a paired 1.1 mutation proving the refusal fires PLUS a negative control proving six live shipped entries still ACCEPT so the fix is not a 11.4.201(1) false-positive refusal.

ATM ID	Type	Status	Severity	Description
BOB-188	Bug	In progress	—	Discovered out-of-band during T028 remediation and reported by the agent as needing tracking; it is also a 11.4.238 coverage escape. OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): UNTRACK THE BINARY — BUILD ON DEMAND §11.4.30 and recorded as consumer DATA: the compiled workable-items binary is NOT to be git-tracked. ‘Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned’ applies without exception here. Keeping it tracked with only invariant-52 fingerprint gate, and keeping it tracked with a commit-time rebuild, were both offered and NOT chosen. REQUIRED END STATE: the gate builds from source, or REFUSES HONESTLY when the Go toolchain is absent (§11.4.201(11) — probe the artifact through its real invocation path, never fake a pass, never silently fall back to a stale binary). Staleness becomes structurally impossible rather than merely detected. ACCEPTED COST: a fresh clone needs a Go toolchain before invariant 17 can be reached — already true for the qBitTorrent-go backend so this adds no new host requirement. The invariant-52 fingerprint gate remains useful for defence-in-depth for as long as any binary exists on disk. This answer is recorded as consumer DATA per §11.4.35 — it is the operator’s stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applies to decisions). — prior item text follows — pre_build_verification.sh invariant 17 (CM-WORKABLE-ITEMS-VALIDATE) resolves its binary through the candidate loop at :534, with FIRST entry is constitution/scripts/workable-items/bin/workable-items. That file is GIT-TRACKED (md5 17644a248363, identical to bin/workable-items-linux) and executable, so it V

ATM ID	Type	Status	Severity	Description
				resolution over the current untracked sibling constitution/scripts/workable-items/workable-items (md5 43376a6d0184). The tracked binary STALE: it does not contain the guards its own source now has. MEASURED, needle-proven, 2026-08-25. String presence in the tracked binary “refusing to set terminal status” -> 0; “Issues-location item has TERMINAL status” -> 0; compare needle from the SAME updateCmd, “at least one mutable field flag is required” -> 2 (so the instrument sees through that path and the zeroes are real absences, not a blind read). The untracked sibling returns 1 / 1 / 2 for the same three queries. A check whose message string is absent from the binary cannot fire. RUNTIME PROOF: the same injected violation (a row set terminal status while current_location=Issue) run through both binaries -> stale reports 1 violation (only the body_md desync class), current reports 2 including the location-status class verbatim. This is exactly why the ten BOB-166 rows (BOB-087/129/131/144/145/146/148/153/155/157) survived undetected: the gate that was supposed to catch them was running a binary structural incapable of seeing them. The 11.4.108 SOURCE >ARTIFACT gap: source correct, artifact stale, every gate reading the artifact reports green. This is the THIRD instance of that class found in one session, alongside BOB-169 (a charset gate that never opened a file) and BOB-183 (a service bundle five days older than its sources). COMPOUNDING: the comment at :519-523 directly above the loop asserts “The naive binary workable-items path never existed in this checkout (bin/ is a gitignored local build-output dir nothing ever populated)”. That statement is now FALSE - bin/ holds two tracked binaries. stale comment asserting the absence of the v file that now shadows resolution is how this stayed invisible. Also an 11.4.30 question the operator owns: bin/workable-items and bin/workable-items-linux are VERSIONED BUILD

ATM ID	Type	Status	Severity	Description
				ARTIFACTS in a constitution submodule that inherited by reference by every consumer, so every consumer inherits whichever binary was last committed. Flagged independently by two separate agents this session. ACCEPTANCE: (a) resolution must never prefer a stale artifact over a current one - either the tracked binaries are removed and resolution falls through to build on-demand, or a freshness check refuses a binary older than its sources (see the BOB-18 fingerprint gate for a working pattern); (b) the false comment at :519-523 corrected; (c) a pair 1.1 mutation proving the new refusal fires, plus negative control proving a genuinely fresh binary still passes so the fix is not an 11.4.201 false-positive refusal; (d) the 11.4.30 tracked-binary decision recorded as operator DATA. Discovered out-of-band during BOB-166 remediation - a 11.4.238 coverage escape in its own right. WHAT: the fail-open scanner's shape-(A2) heuristic cannot distinguish 'return False' meaning PROCEED-AS-IF-FINE from 'return False' meaning REFUSE. It flags six textbook 1 CLOSED guards as fail-open defects: <code>_is_safe_fetch_url</code> (x3), <code>_qbit_add_succeeded</code> (x2) and the hooks path-boundary guard. INDEPENDENTLY VERIFIED 2026-08-25 by the conductor rather than taken on the triage agenda. word: <code>download-proxy/src/api/routes.py:112</code> defines <code>_is_safe_fetch_url</code> returning bool, and only call site at :1476 reads 'if not <code>_is_safe_fetch_url(url)</code>: <code>logger.warning(Refused SSRF-unsafe download URL (non-public target skipping); continue</code>' - a False return REFUSES URL. WHY THIS MATTERS: per §11.4.201(1) a false-positive refusal is a FAIL-bluff of equal severity to a false-negative pass, and here the consequence is worse than noise - acting on the finding would mean making the SSRF guard fail by returning False, i.e. deleting an SSRF protection to satisfy a gate. A gate that instructs you to
BOB-189	Bug	Queued	—	

ATM ID	Type	Status	Severity	Description
BOB-191	Bug	Queued	—	remove a security control is actively dangerous, not merely imprecise. REPRO: run the fail-open scan; observe six hits; read each call site. ACCEPTANCE: the scanner distinguishes refusal-shaped from proceed-shaped False returns (call-site-aware, or an audited waiver list with per-entry justification), the six drop out, AND a golden-FALSE fixture containing a real fail-closed guard is added so the discrimination is itself falsifiable per §1.1. WHAT: the §11.4.252 fail-open scan reports 0 for qBitTorrent-go, and that zero is NOT evidence. The triage agent ran control needles through the scanner's own path per §11.4.207 (b): a Python 'except Exception: pass' needle WAS SEEN, a TypeScript 'catch (e) {}' needle was SEEN (so frontend/src's zero IS a real zero), but a Go empty-'if err != nil {}' needle was NOT SEEN. The instrument that cannot see the idiom returns the same quiet zero as a clean tree, and only one of those is honest. DISTINCT FROM BOB-189, BOB-189 is a false MATCH (fail-closed guards reported as fail-open); this is a false NULL (an entire language unscanned). Same scanner, opposite failure directions, different fixes - merging them would hide one behind the other. IMPACT: Go is the language of qbittorrent-pro, qBitTorrent-go and boba-jackett (port 7189, which owns encrypted tracker credentials), so the unscanned surface is exactly where §11.4.252's credential-plus-mutation combination is most likely. Nobody has assessed it; the dashboard says clean. ACCEPTANCE: the scanner grows a Go tree covering the empty-err-block and swallowed error idioms, its needle is SEEN through the scanner path, qBitTorrent-go's result is re-derived, and every finding is triaged as this Python/TS pass was. Until then qBitTorrent-go's fail-open position is UNKNOWN and must be reported as UNKNOWN, never as 0. === ROUND-2 INVESTIGATION, MEASURED 2026-08-26 — T

ATM ID	Type	Status	Severity	Description
				FILED TITLE WAS TOO NARROW (§11.4.6) == The live scanner is constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh, driven per-root by invariant 39 at scripts/pre_build_verification.sh:1481-1544. (scripts/pre_build/check_cm_no_fail_open_skip.sh has NEVER existed on any branch — it exists only on an uncommitted worktree. Any citation of the path is citing a file that is not there.) IT IS A WEAK-MATCHER DEFECT, NOT A SCOPE HOLE Line :243 sets exts=“py go rs c cc cpp h hpp java cs js ts jsx tsx php rb” — .go IS enumerated. But line :537 gates the Python ast analyser on cases "\$f" in *.py), so every non-Python file falls through to just two text matchers (:772 empty catch (...) {, :794 credential = x \\ \\ "literal"). Go has no catch keyword, so neither matcher CAN fire on Go by construction. NEED TEST, BOTH ARMS (planted / rc / result): Python 2 / 1 / SEEN · TypeScript 1 / 1 / SEEN · Java 1 / 1 / SEEN · Go 6 / 0 / NOT SEEN — PASS · Rust 2 / 0 / NOT SEEN · Ruby 1 / 0 / NOT SEEN · C 2 / 0 / NOT SEEN. Go is ONE OF FIVE BLIND EXTENSION MATCHES, not a special case. The Go needle is real code: build rc=0, go vet rc=0, gofmt -e clean. THE DECISIVE DISCRIMINATOR — same bytes, extension changed: .zig (NOT enumerated) → “SKIP — topology_unsupported”; .go (enumerated) → “PASS”. The gate ALREADY OWNS an honest-blindness mechanism and it works correctly; Go routes around it precisely BEING ENUMERATED. A scope hole announced itself; this matcher hole prints green. That makes it the worse defect and is why the corrected test leads with it. QUANTIFIED SURFACE (exclusion stated, §11.4.224(E)): using the scanner’s OWNED prune list and nothing added — 106 .go files / 20,389 LOC (52 production files / 6,858 LOC). WITH and WITHOUT exclusions the count is 106 == 106; no vendored tree exists, so no unstateful exclusion could manufacture a clean number. qBitTorrent-go holds 106 blind files and 0 analysable files — 100% blind — yet invariant

ATM ID	Type	Status	Severity	Description
				counts it among “5 first-party source root(s)” reported clean. DEFECTS THE BLINDNESS HOLE → filed as BOB-204 (credential DELETE returns unconditional 204 while the .env plaintext de error is discarded; plus the env_write_failed_db_rolled_back code asserts an unconfirmed rollback). SEPARATE SCOPE HOLE → filed as BOB-205 (cmd/boba-ctl, 947 orchestrator, absent from DANGER_ROOTS entirely). Distinct-but-similar per §11.4.214, deliberately not merged. GOLDEN-FALSE FIXTURE FOR ANY FUTURE GO ARM: internal jackettapi/auth_middleware.go:45-66 is correct fail-CLOSED (constant-time compare, 401+ret — a Go arm MUST NOT flag it. LOAD-BEARING §11.4.250 PREDICTION: bolting a naive Go reg arm on will IMMEDIATELY manufacture BOB-189-class false positives in Go — if err nil { return false } in a Go validator is fail-CLOSED. Fixing this as layer N+1 reproduces BOB-189 in a new language. THEREFORE the remediation order is fixed: (1) analyser REGISTRY with an UNANALYSED verdict FIRST extension→analyser map, unmapped extensions reports UNANALYSED never silent PASS, which converts the false null into an honest gap and covers rust/ruby/C too; (2) only then a Go arm call-site-aware from day one (go/ast, or adopted errcheck/staticcheck rather than hand-rolled regex), shipped with the auth_middleware golden-FALSE. ONE PRIMITIVE, TWO SYMPTOMS: the scanner classifies by local syntactic shape and never consults semantic — BOB-189 ignores the CALLER’s context (false match), BOB-191 searches for a shape from a DIFFERENT language family (false null). The gate’s own header already concedes the primitive (“requires real data-flow analysis this gate CANNOT honestly claim”); what is missing is propagating that concession into PER-LANGUAGE honesty. Keep two tracker items. INSTRUMENT BUG RECORDED, NOT HIDDEN (§11.4.201(12)) the investigation’s first verdict extraction use

ATM ID	Type	Status	Severity	Description
				grep -oE 'PASS\ FAIL\ SKIP', which matched FAIL inside the gate's own name (...FAIL-CLO and mislabelled three PASSes. Re-run keyed exit code; the table above is the corrected run EVIDENCE: docs/qa/BOB-191/scanner_blindness_investigation_20260826.m (255 lines, §11.4.44 header). No source, gate, config modified by the investigation. WHAT: the BOB-161 gate lands with 6 real fail open skips RATCHETED rather than fixed. Ratcheting is the constitution's named brownfield default (§11.4.135/§11.4.224(E)) and this repo's own precedent, so the choice is correct - but the remediation it defers is real work that must be owned somewhere. WHY THIS ITEM EXISTS: the gate's own header asserted the 6 were 'TRACKED SEPARATELY (§11.4.197)' when no tracker row existed. The §11.4.209 independent review verified the absence and raised it as IMPORTANT-5, noting that without row those findings are precisely the parked-unverified debt class §11.4.226(4) names - the population an operator samples and finds broken. A prose claim of being tracked is not tracking. WHAT EACH NEEDS: a skip that fires evidence the host ANSWERED must either classify the response and FAIL on it, or take a §11.4.69 reason that is honestly derivable from the environment rather than from the response verified against the live stack, not asserted. Two of the six sit under '# allow-skip:' markers at tests/unit/test_tracker_auth_live.py:105 and :106 which the reviewer confirmed genuinely are open, so that marker must not be treated as absolution. ACCEPTANCE: all 6 remediated with RED-first evidence per §11.4.115, the gate's BASELINE ratcheted to 0, and the ratchet's monotone-decreasing property preserved throughout (§11.4.227(A)). NOTE the reviewer MINOR-1: a count-baseline absorbs a one-out one-in swap, so remediation progress must be
BOB-192	Task	Queued	—	

ATM ID	Type	Status	Severity	Description
				checked against the finding SET, not only the count.
				WHAT: in download-proxy/src/api/streaming the dedup set is written BEFORE the yield — seen_hashes.add() / seen_hashes_local.add() execute at :317 and :356 ahead of emitting. If format_event raises, the result is never sent A its hash is already recorded, so it is permanent. dropped. Independently verified by the §11.4 reviewer at both sites: mid-search the hash persists across subsequent polls so the result never re-emits; at completion the stream breaks immediately after, so there is no later chance either. Results later in the SAME batch are not added and do re-emit on the next poll, which is why the observed symptom is partial loss rather than total. WHY IT IS FILED SEPARATELY: the §11.4.252 fail-open pass made this loss OBSERVABLE by adding a log line, which is the right first move — but observability is not repair. The underlying at-most-once semantics remain and per §11.4.226(5) a mitigation cannot close the defect it mitigates. THE DECISION IS A REAL TRADE, not an oversight to correct blindly: moving to add-after-successful-yield gives at-most-once, but a DETERMINISTIC serialization failure would then retry the same result on every poll forever — trading silent loss for a hot local Options are (a) keep at-most-once and treat the log line as the contract, (b) add-after-successful yield with a bounded per-hash retry budget, (c) add-before-yield but remove the hash on emission failure so exactly one retry occurs. ACCEPTABLE an explicit decision recorded, implemented, and covered by a test that drives a raising emit and asserts the chosen semantics — with a RED flag observed first per §11.4.115. Raised as MINOR by the independent review of the fail-open bug filed rather than absorbed so the residual defect is not retired along with the logging that revealed it.
BOB-193	Bug	Queued	—	
BOB-194	Bug		—	

ATM ID	Type	Status	Severity	Description
		In progress		WHAT: agent worktrees under .claude/worktrees/ are ephemeral copies pinned at arbitrary commits - never built, never shipped, never deleted when the agent finishes. Pre-build gates that walk the tree with find(1) from the repo root can't descend into them anyway. MEASURED INSTANCE (2026-08-25): CM-GO-TOOLCHAIN-MATCHES-BUILDER FAILED the main build with exactly one finding, from .claude/worktrees/agent-afda1df906c537ab4 - a worktree 86 commits behind still carrying 'FROM golang:alpine' against a go.mod long since at 1.26.2. The main tree was CORRECT throughout. Per §11.4.201(1) a false-positive refusal is a FAILURE of equal severity to a false pass: it blocks real work and teaches people to bypass the gate. FIXED FOR ONE GATE: '.claude' added to that gate's PRUNE_DIRS with a deterministic \$1.1 - a stale mismatch inside .claude/worktrees must NOT fail a healthy tree, the SAME mismatch outside it MUST still fail, so the prune cannot widen into blindness (§11.4.201(6)). Mutation verified: reverting the prune fails 2 checks. SURVEY, measured not inferred: 8 of 13 pre-build gates walk the tree; only the fixed one prunes .claude. Gates using 'git ls-files' are structurally immune - worktrees are untracked. Empirically probed: check_cm_closure_seam_binds, check_cm_killpg_pgid_guard and check_cm_no_production_mutation_residue all rc=0 with ZERO worktree references - clean in practice today. UNMEASURED, not clean: check_cm_test_mock_pid_explicit_int and check_cm_test_mock_pid_patched_when_real were not reached before the probe budget expired. HYPOTHESIS REFUTED, recorded so nobody re-walks it: an earlier revision of this item speculated that check_cm_plugin_count 90s timeout was caused by walking worktree is NOT. That gate's find is scoped to \$PLUGINS_DIR (check_cm_plugin_count.sh:2) so worktrees are outside its walk entirely. Its

ATM ID	Type	Status	Severity	Description
BOB-195	Bug	In progress	—	slowness is real - still running past 54s on a r measure - but has a different, still-unidentifie cause and belongs to its own item, not this or ACCEPTANCE: every tree-walking gate either prunes agent scratch or is proven immune by measurement, each with a paired mutation. NOTE the asymmetry that bounds the risk: a worktree can only ADD findings to a security relevant scan (killpg, mutation-residue), neve remove them - so the exposure is false refusa and eroded trust, not a missed defect. OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): TEACH THE SCANNER With NODES — KEEP SIM105 The §11.4.252 fail-open scanner is FIXED to see contextlib.suppress; ruff's SIM105 rule STAY ENABLED in pyproject.toml. Disabling SIM10 and the belt-and-braces both-at-once option, were offered and NOT chosen. Rationale carr with the decision: fixing the gate makes SIM1 harmless, whereas disabling SIM105 alone w leave the gate blind to any suppress written b hand, inherited from a dependency's style, or already present in the tree. REQUIRED BEHAVIOUR: with contextlib.suppress(...) with suppress(...) (the from contextlib imp suppress binding) wrapping a dangerous- combination call are DETECTED with the sam severity and message shape as the try/except pass form; a narrow suppress(SpecificError around a NON-dangerous call must NOT fire, since a false-positive refusal is a §11.4.201(1) FAIL-bluff of equal severity to a missed real o CONSEQUENCE FOR THE NUMBER: once land the scanner's finding count stops being a floo over try/except shapes and becomes a genuin census — any prior count citing completenes was scoped to try/except only. This answer is recorded as consumer DATA per §11.4.35 — i the operator's stated choice, not an agent inference, and supersedes any prior agent- chosen default on this question. Options not

ATM ID	Type	Status	Severity	Description
				chosen are named above so a future reader c not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisio — prior item text follows — WHAT: the §11.4 fail-open scanner matches Try-handler shape (A1)/(A2). contextlib.suppress is a With node, swallow written that way is invisible. VERIFI BY THE CONDUCTOR with a control needle through the same invocation path (§11.4.201 (b)), because a first attempt returned 0 for BC forms and was itself blind - the gate takes -ro not a positional, so a bare path exits on an unknown arg (§11.4.201(7)(c): the invocation path is part of the instrument). Correct run: t except/pass around os.unlink(user_supplied_path) -> 'FAIL - swallowed exception ... :5', the needle sees. contextlib.suppress(Exception) around the IDENTICAL call -> 'PASS - no swallowed- exception ... anti-patterns found'. Both combi untrusted input with an irreversible unlink; swallow everything; one is caught and one is WHY IT IS WORSE THAN A PLAIN GAP: pyproject.toml enables ruff's SIM ruleset, and SIM105 is 'use contextlib.suppress instead of except-pass'. Our own linter therefore instr authors to rewrite the form this gate CAN see into the form it CANNOT. The two tools are pulling in opposite directions and the lint one runs more often. Every SIM105 autofix silent shrinks this gate's coverage while the count g down, which reads as progress. CONSEQUEN FOR THE NUMBER: the scanner's finding cou a FLOOR over try/except shapes, not a census swallows. Any statement of the form 'N fail-o sites remain' must be read as 'N among try/ except shapes'. ACCEPTANCE: the scanner recognises contextlib.suppress (a With whose items call contextlib.suppress with a broad exception type) and applies the same ≥ 2- capability test; a golden-TRUE fixture in supp form; a golden-FALSE fixture where suppress wraps a single-capability diagnostic emitter a

ATM ID	Type	Status	Severity	Description
BOB-196	Task	Queued	—	must NOT fire; and an explicit decision on the SIM105 tension - either exempt the shapes the gate governs, or accept suppress and teach the gate to read it. Discovered by the §11.4.252 remediation agent when its own MINOR-3 fix required using contextlib.suppress at three diagnostic-emitter sites; those three uses are legitimate and justified in-source. WHAT: the CM-PLUGIN-COUNT pre-build gate takes 189 SECONDS to verify 8 documented counts. Measured 2026-08-25: exceeded a 90s probe budget (rc=124), then a bounded re-run completed at 'rc=0 wall=189s' - it PASSES, it is simply slow. CAUSE, MEASURED NOT GUESSED the marker loop at check_cm_plugin_count.sh:273-290 iterates EVERY LINE of every governed document and spawns a pipeline per line. Per iteration: 'priority grep -oE wc -l tr -d' (4 processes) plus 'priority sed -n' (2) = ~6. Governed docs are AGENTS.m 689 + CLAUDE.md 494 + docs/features/Status. 670 = 1853 lines, so ~11,118 process spawns, measuring ~102 ms/line. Classic per-line-subprocess shell trap. THE IRONY WORTH PRESERVING, because it explains why nobody simplified it: the in-code comment shows 'only wc -l' was chosen DELIBERATELY over 'grep -oE' after measuring the §11.4.201(12) footgun - on this host (ugrep 7.8.4) 'grep -coE' returned 3 at top level but 1 inside a 'set -euo pipefail' subshell a context-dependent reading. The correctness is right and must be kept; it is the PER-LINE application of it that costs the three minutes. HYPOTHESIS REFUTED, recorded so nobody walks it: this is NOT worktree traversal. The gate's find is scoped to \$PLUGINS_DIR (:224), the five agent worktrees are outside its walk. That guess was raised on BOB-194 and is dead. LIKELY FIX (unverified, stated as a candidate a conclusion): pre-filter with ONE grep for lines containing 'CM-PLUGIN-COUNT:' before entering the loop, reducing ~1853 iterations to the ~8 lines

ATM ID	Type	Status	Severity	Description
				that carry a marker, while leaving the per-line parsing logic - and its footgun-avoiding form completely unchanged. WHY IT MATTERS beyond impatience: this gate is on the pre-bu critical path, and CLAUDE.md leans on it as the mechanical authority for the three distinct plugin rosters (43 curated / 43 engines / 12 bootstrap) whose conflation was BOB-149. A slow enough to tempt anyone into skipping it protects nothing. ACCEPTANCE: runtime reduced with the counts and the footgun-avoiding preserved byte-for-byte in behaviour, proven the existing paired mutation still biting plus a before/after wall-clock recorded. §11.4.6: not here questions the gate's VERDICTS - only its runtime. OPERATOR DECISION (2026-08-26, §11.4.66) GENERATE THE TOKEN AND ARM IT The conductor generates a 32-byte random token, writes BOBA_API_TOKEN into .env; the operator configures clients with it. DONE 2026-08-26: token generated via secrets.token_hex(32), appended to .env with a comment naming BOB-197 and the routes.py:83 env-gating fact chmod 600, value shown to the operator ONCE and never written to any other file. Safety verified BEFORE writing (§11.4.30/§11.4.10): .gitignore:27 '*.env' and was untracked (needle-proven — the same matcher confirmed docker-compose.yml IS tracked, so the negative is real); a §9.2 pre-op backup was taken and is ignored; post-write re-check confirmed .env is ignored and absent from git status. Pre-store audit found the only tracked BOBA_API_TOKEN occurrences are the docker-compose passthrough \${BOBA_API_TOKEN:-} and proposed the leak-audit challenge — no literal value has ever been committed. STILL OWED, and the i stays open until it lands: the BOOT-TIME INVARIANT (§11.4.254) that refuses to start L bound when the token is unset or empty. With it the arming is a state fix, not a defect fix —
BOB-197	Bug	In progress	—	

ATM ID	Type	Status	Severity	Description
				§11.4.226(5) is explicit that a state-only repair cannot close a defect item. Options not chosen by the operator sets it themselves with the invariant blocking until then; invariant warns first and blocks on a named date. Recorded as consumed DATA per §11.4.35 — the operator's stated choice is not an agent inference. Options not chosen are named so a future reader does not re-litigate a settled call. --- prior item text follows --- OPERATOR DECISION (2026-08-26): arm BOBA_API_TOKEN and keep 0.0.0.0, backed by BOOT-TIME invariant that refuses to start LAN-bound with the token unset. This item IS that invariant. MEASURED FACT, not inference. download-proxy/src/api/routes.py:83 require_api_token() reads BOBA_API_TOKEN at request time and returns immediately when unset or empty. Its own docstring states it plainly: "BOBA_API_TOKEN unset/empty -> return (OPEN). This is the DEFAULT and preserves the current no-auth contract + dev workflow". docker-compose.yml:237 passes BOBA_API_TOKEN=\${BOBA_API_TOKEN:-}, so the container inherits empty unless the operator declares it. The operator .env does NOT declare it, needle-verified per 11.4.201(7)(b): the same matcher hit QBITTORRENT_DATA_DIR in the same file, so the miss is a real absence and not a blind read. The qbittorrent-proxy container was RUNNING and LAN-bound at measurement time. Consequence: the ten routes the BOB-102 gateway resolves as auth-wired are open in practice on default deploy. WHY A STATIC GATE CANNOT COVER THIS. The BOB-102 gate asserts static route WIRING - that a Depends(require_api_token) marker exists on a handler, decorator or router. It cannot assert runtime ARMING, which depends on an env read per request. Claiming the static gate covers it would be exactly the bluff the gate exists to prevent (11.4.262: machine evidence must match the layer it claims). This is a boot-time invariant class per 11.4.254. ACCEPTANCE: a boot-time

ATM ID	Type	Status	Severity	Description
BOB-198	Bug	Queued	—	check that, when the listener is LAN-bound (loopback), refuses to start with a non-zero ex and a named cause if BOBA_API_TOKEN is un or empty, so the open state becomes UNREACHABLE rather than the default. Paired 1.1 mutation: remove the check, prove the service starts LAN-bound-and-open. Golden-FALSE per 11.4.201(1): a loopback-bound listener with the token unset must NOT be refused, or check becomes a false-positive refusal blocking legitimate dev work. 11.4.238 COVERAGE-ESC NOTE: found by a subagent reading source during BOB-102 guard construction, NOT by the automated QA regime - which is itself the default class 11.4.238 names. The boot-time check IS new automated check that would have caught OPERATOR DECISION (2026-08-26, §11.4.66) REFUSE TO START LAN-BOUND The Go process gets a boot-time guard: bind loopback only, or refuse to start. It does NOT get auth middleware in this cycle. Rationale carried with the decision: the profile is opt-in via -profile go, was not running when measured, and is used for development — so closing the exposure costs a guard rather than a parity implementation. Options not chosen: write auth middleware to parity with the Python route set (real work, and it would partially un-defer BOB-101); document dev-only with no guard (rejected — nothing would enforce it, and the next -profile go runs this network silently re-opens 22 routes). ACCEPTANCE: a boot-time check that resolves listener's bind address and refuses to start with it is not loopback, with a paired §1.1 mutation proving the check FAILs on a 0.0.0.0 bind, and golden-FALSE proving a loopback bind is NOT refused (§11.4.201(1)). Scope reminder recorded so it is not lost: this is NOT covered by BOB-10 parity deferral, which was about ports; folding in would be a §11.4.112(5) verdict leak. Recorded as consumer DATA per §11.4.35 — the operator stated choice, not an agent inference. Options:

ATM ID	Type	Status	Severity	Description
				chosen are named so a future reader does not litigate a settled call. — prior item text follows MEASURED, conductor-verified independent the reporting subagent. qBitTorrent-go/cmd/qbittorrent-proxy/main.go registers exactly the middlewares, at lines 62, 63 and 68: middleware.CORS, middleware.Logger, middleware.GinRateLimit. There is NO authentication middleware and no per-route auth dependency. main.go:119-121 binds addr=":" which is ALL interfaces, so every router is LAN-reachable when the profile runs. SCOPE 22 routes, of which the mutating ones matter most - POST /api/v1/download, POST /api/v1/magnet, DELETE /api/v1/hooks/:id, POST and DELETE /api/v1/schedules. A LAN peer can query downloads and delete webhook configuration with no credential. PARITY GAP, stated as fact PUT /api/v1/theme is auth-gated in the Python service and open in Go. The same asymmetry holds for the download, hooks and schedules mutations. Two implementations of one advertised surface disagree about whether it needs a credential. RELATIONSHIP TO BOB-1 (Go parity deferred past v1.0.0, operator decision 2026-08-26): that deferral covered PORT parity the Go container binding :7186 and :7188 as well as :7187. It did NOT cover an authentication hole and the operator was not asked about one, because the hole had not been measured when the question was posed. Treating this as covered by the deferral would be a 11.4.112(5) verdict leak: citing a bounded decision outside the scope of its evidence supports. This item stands separately and its severity is NOT settled by BOB-101. MITIGATING FACT, so severity is not overstated the Go profile is opt-in via -profile go and was NOT running at measurement time (the container list showed qbittorrent-proxy, the Python service, with no Go sibling). The exposure is latent, not live. ACCEPTANCE: either authentication middleware landed in the Go service with passage to the Python route set, or an explicit operator

ATM ID	Type	Status	Severity	Description
				decision recording the Go profile as dev-only must refuse to start LAN-bound - with a guard enforcing whichever is chosen. OPERATOR DECISION (2026-08-26, §11.4.66) FLAG NARROW SUPPRESS ONLY WITH AN IRREVERSIBLE CAPABILITY The scanner flag narrow contextlib.suppress ONLY when combined with an irreversible capability (delete / truncate / kill) — the shape that actually causes harm. Idiomatic narrow tolerances stay quiet; the gate keeps its credibility and no false-positive storm trains readers to ignore it. Options not chosen: accept the asymmetry permanently via a header statement; flag all narrow suppressions; absorb existing sites via a justified exemption list. MEASUREMENT CORRECTED TWICE, and the second correction explains the first: the original ‘37 narrow sites, all idiomatic’ was propagated by this conductor without verification, then re-measured by the authoring agent as OBSERVED CONTAMINATION (§11.4.201(10)) — 27 vendored third-party + 9 sibling-agent worktree COPIES of the very tree being measured + 1 real = 37. The instrument was counting other agents’ duplication of its own subject. Reproducible figures, each with its scope stated: constitution repo 0 narrow with-suppress; boba DANGER_ROOTS 0 narrow with-suppress and 23 narrow except-pass; boba repo minus vendored 1 narrow with-suppress (suppress(OSError)); narrow except-pass RETRACTED-AND-RESTATED 2026-08-26 per §11.4.201(9): the figure 97 previously recorded here reproduces under NO scope and is withdrawn. Measured replacements, each with its scope and both independently reproduced: narrow except-pass git-tracked first-party (canonical); 50 narrow except-pass on-disk minus scratch dirs (the ‘minus vendored’ scope this sentence states). Brackets that explain the balance number: 87 ALL-except-pass git-tracked and 87 ALL-except-pass on-disk-minus-scratch straddled 97, so 97 was a class-scope conflation (narrow
BOB-199	Task	Queued	—	

ATM ID	Type	Status	Severity	Description
				all), not a corpus-scope one. The 35 cited later this record is the same measurement at git-tracked scope and agrees with 34 within one. The ‘every one idiomatic’ claim was also false. The except-pass census contains SystemExit x (tests/unit/test_plugin_torrentscsv.py:305,328) which is not a benign tolerance. CORRECTION 2026-08-26: a ‘KeyboardInterrupt x1’ previously written in this sentence was itself unsourced. This is WITHDRAWN — it reproduces at NEITHER stated scope. Independent AST count over git-tracked *.py: ZERO genuine ‘except KeyboardInterrupt: pass’. The only genuine instance in the checkout is .worktrees/ci-split-workflows/tests/unit/test_main.py:63 — a sibling agent scratch worktree, i.e. the exact observed contamination class (11.4.201(10)) this very record teaches to exclude — and the only match tree textual match (tests/unit/test_graceful_shutdown.py:165) sits inside a triple-quoted string literal, a carrier not code (11.4.201(7)(a)). Recorded because the error is instructive: it was introduced BY the retraction that removed the 97, so a correction is not exempt from the discipline it applies. Recorded as consumer DATA per §11.4.35 — the operator stated choice, not an agent inference. Options chosen are named so a future reader does not litigate a settled call. — prior item text follows RESIDUAL ASYMMETRY surfaced by the BOB fix, reported rather than silently closed. After teaching the scanner With nodes, a BROAD suppress (Exception / BaseException) is detected with the same severity as try/except/pass. A NARROW suppress (suppress(FileNotFoundError)) is not - but the semantically identical narrow try/except FileNotFoundError: pass IS flagged. So ruff SIM105 rewriting a narrow handler still moves that site out of the gate’s scope, a smaller version of the exact mechanism BOB-195 was filed to close. WHY IT WAS NOT CLOSED IN THAT PAPER with the measurement that decided it: the

ATM ID	Type	Status	Severity	Description
				authoring agent reported 37 narrow suppress sites, all idiomatic. CORRECTION (2026-08-26, independent review): that figure DOES NOT REPRODUCE and this conductor propagated it into this item as fact without verifying - the error is mine, not the reviewer's. Measured: the constitution repo has 0 narrow with suppress sites; boba first-party has exactly 1 (a suppress(OSError)). The nearest corpus match is 35 narrow except X: pass handlers in boba first-party - a DIFFERENT construct - and their class census (OSError x7, ImportError x7, BrokenPipeError x3, RuntimeError x3, ValueError x2, FileNotFoundError x2, IndexError x2, SystemExit x2) contradicts 'every one idiomatic': a bare SystemExit: pass is not a benign tolerance. The two populations must not be conflated, and the distinction is the whole substance of this item. The false-positive-storm argument therefore rests on the 35 narrow except-handlers, not on 37 suppress sites, and its strength should be re-judged on that basis, with the result that 11.4.201(1) is a FAIL-bluff of equal severity to the gap it would close, and worse in practice because it trains readers to ignore the gate. The subagent recorded the asymmetry in the gate header as a known gap rather than shipping the storm. That was the right call, and is why this item is a separate tracked item rather than an unfinished one. THE DECISION IS THE OPERATOR'S, mirroring BOB-195 itself: (a) accept the asymmetry permanently, with the gate header stating it so no reader mistakes the count for a census; (b) flag narrow suppress ONLY when combined with an irreversible capability (delete / truncate / kill), catching the shape that actually matters and leaving the 37 idiomatic sites quiet; (c) flag all narrow suppress and absorb the 37 into a justified exemption list like the LAN-route guard uses. RELATED FACT worth carrying: the five newly-visible production sites in download.proxy/src/merge_service/search.py (1294, 1300, 1322, 1329, 1333) wrap proc.kill / os.killpg /

ATM ID	Type	Status	Severity	Description
BOB-200	Bug	Queued	Medium	proc.wait in the BOB-126 cleanup path. The conductor verified the 11.4.263 pgid guard IS present and correct at both killpg sites - _pid _pgid each checked isinstance(int) and > 1 before the syscall, with the BOB-126 forensic reason inline - so those sites are true-by-the-scanner definition but SAFE in fact, and want a justification exemption entry rather than a code change. WHAT: scripts/gates/cm_dangerous_combination_fail_closed.sh by its scan file list with find(1). If find itself DIES (permission error, resource exhaustion, interrupted), the gate receives an EMPTY file and interprets that as 'no files in scope' -> ho topology SKIP -> exit 0. The failure IS loud on stderr but SILENT in the exit code, so any call gating on exit status reads a dead instrument clean corpus. WHY IT MATTERS: this is a textbook 11.4.201(6) FALSE-NULL – a blind instrument and a clean artifact return the identical quiet zero. It is also a 11.4.252 fail-O on exactly the 'cannot enumerate the corpus' condition where the gate's own stated discipline is to fail CLOSED. A gate that cannot see must refuse, not pass. AFFECTED SCOPE: constitution scripts/gates/cm_dangerous_combination_fail_closed.sh (the find invocation and the empty-list branch). PRE-EXISTING – NOT introduced by the BOB-195 change; found by the independent round-2 reviewer while reviewing that change and explicitly scoped OUT of its remediation. REPRODUCTION: make find(1) fail during the gate run (unreadable scan root, or a find stuck returning non-zero with empty stdout) and observe the gate exit 0 with a topology-SKIP message, indistinguishable from a genuinely empty corpus. ACCEPTANCE: (1) the gate distinguishes 'find succeeded and found zero files' from 'find failed' – check find's exit status not only its output; (2) a failed enumeration is FINDING (non-zero exit) naming the unresolved

ATM ID	Type	Status	Severity	Description
BOB-201	Bug	Queued	Medium	precondition, never a SKIP; (3) a genuinely empty scan root still SKIPS honestly at exit 0 - 11.4.201(1) golden-FALSE guard, so the fix does not become a false-positive refusal; (4) paired mutation: restore the swallow-find-failure behaviour and the new fixture MUST fail. WHAT: scripts/ownership_repair.sh fences declared scope entries LEXICALLY (string normalisation + prefix/traversal refusal). A lexical fence cannot see a symlink sitting at an INTERMEDIATE path component. A static (already-present, no race required) symlink in the middle of an otherwise-legal declared path steers the recursive walk at a tree the operator never declared. Measured by the T028 round-trip independent reviewer as surviving mutation. WHY IT IS FILED SEPARATELY: the in-source honest-boundary note and Case 24 fixture assume this reach is 'tracked as BOB-159'. It is NOT. VERIFIED 2026-08-26 by direct query: BOB-159 description is 4685 chars with ZERO occurrences of 'symlink' or 'intermediate' (control-needle in the body is readable through the same query path), and no item in the tracker recorded the reach at all. A dangling tracker citation means the real defect is recorded NOWHERE - the 11.4.201 lost-defect shape, where the pointer looks like coverage and is not. This item IS that record; the in-source note must now cite THIS id. SCOPE SEVERITY BOUNDS, stated honestly (11.4.6): the reach is BOUNDED, not arbitrary. It requires an attacker or accident to have already placed a symlink inside a declared scope path. It is NOT reachable from the shipped config/owned_paths.yaml as it stands (shipped-scope control needle: 6 rows, RC=0). It is a real widening of what a recursive chown can touch, not a theoretical one - the reviewer measured no race needed. WHAT THIS ITEM DOES NOT CLAIM: that the lexical fence is broken. The fence does what it says - it refuses lexical escapes ('.' system trees, '..' climbs) and was verified doing

ATM ID	Type	Status	Severity	Description
				under 12 reviewer-authored mutations. This documented LIMIT of lexical fencing, now recorded as a real item instead of a citation to unrelated one. ACCEPTANCE: (1) decide deliberately and record the decision - resolve components (realpath/-P semantics) and re-fence the resolved path, OR keep the lexical fence as state the limit as an accepted operator-owned risk; the choice is operator-owned per 11.4.66 because resolving makes the fence depend on filesystem state at check time, which has its own failure modes; (2) if resolution is chosen, a golden-FALSE set proving legitimate symlink but-in-scope download roots still walk (11.4.201(1) - a fence that over-refuses is a FA bluff of equal severity); (3) the in-source note the Case 24 fixture cite THIS item, not BOB-15 (4) paired 1.1 mutation: restore the un-resolved behaviour and the fixture MUST fail.
				WHAT: scripts/ownership_repair.sh:493 reads parsed scope rows with 'while IFS=\$'\t' read -e_path e_kind e_opt e_pres e_rec'. A scope entry that OMITS 'kind' emits the row '<path>\t\t1\t TAB is an IFS-WHITESPACE character in bash even when IFS is set to it explicitly, so a RUN consecutive tabs collapses into ONE delimiter instead of delimiting an empty field. Every field after the omission shifts left by one. MEASURED 2026-08-26 on GNU bash 5.2.37, control-needed (the with-field case was run FIRST and produced output, proving the instrument sees; an earlier probe of mine returned nothing for BOTH cases because printf lacked a trailing newline - an error 11.4.201(7)(b) false-null I discarded rather than reported): kind PRESENT: path=[P] kind=[dir] opt=[1] pres=[0] rec=[1] kind EMPTY: path=[P] kind=[1] opt=[0] pres=[1] rec=[] CONSEQUENCE 'optional' is read out of 'preserve_mode's slot entry declared 'optional: true' is therefore treated as NON-optional, so an absent path that should skip honestly instead becomes a hard failure and 'preserve_mode'/'recursive' are likewise
BOB-202	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				from the wrong slots, with 'recursive' arriving EMPTY. The shift is silent: no parse error, no diagnostic, no refusal. REACHABILITY: NOT reachable from the shipped config/owned_paths.yaml - all six declared entries c 'kind' (verified). This is a latent defect in the reader, not a live misbehaviour of the product today. PROVENANCE: surfaced out-of-band by T028 round-5 remediation agent while isolating an unrelated instrument slip, and deliberately left unfixed and unfiled by it because (a) ownership_repair.sh had to stay at hash ae025b74602414ca for that round's do-not-regress set, (b) it was outside that round's two NIT remit, and (c) it had not root-caused where the repair belongs in the parser, the consumer, or the schema. That judgement was correct; this item is the filing it deferred. Independently measured by the conductor before filing - not accepted on report. 11.4.238 COVERAGE-ESCAPE NOTE: no automated check found this. It was found by a human-directed agent isolating a different problem. Per 11.4.238 the coverage is itself a defect of equal standing to the bug, closing only the bug is the violation. ACCEPTANCE: (1) root-cause the correct layer parser (never emit a row with an empty field consumer (read with a delimiter that is not II whitespace, or read positionally), or schema (make 'kind' mandatory and REFUSE an entry lacking it, consistent with the whole-run refusal already landed for empty expansions); the change is operator-owned per 11.4.66 because it changes the scope-file contract; (2) a fixture with an omitted 'kind' and 'optional: true' asserting the entry is treated as OPTIONAL (or the row refused), RED-first against current code; (3) a golden-FALSE proving a fully-specified row still parses identically - 11.4.201(1), the reader must not start refusing valid scopes; (4) paired 1.1 mutation restoring the collapsing read so the fixture FAILs; (5) an automated check that would have caught it, per 11.4.238. PRECEDENT FOUND

ATM ID	Type	Status	Severity	Description
				2026-08-26 (surfaced by the T028 round-5 reviewer, independently verified before recording): the SIBLING script scripts/ownership_precondition.sh ALREADY documents this exact collapse class at lines 565-578 and already ships the repair as split_tsv() at line 5 (used at 437, 699, 775). Its in-source note carries a real forensic measurement dated 2026-08-21: FIRST version of that script used the collapsing read and reported "no compose service mount at this location" for config/ and for the download root WHILE FIVE SERVICES MOUNT THEM - a false statement about what was checked, produced by the instrument rather than the system, which its own comment classifies as 11.4.201(7)(c) 'the path is part of the instrument failure. It also records why the unit suite could not see it: the fixture scope has no compose service at all, so only running the real invocation against the real scope surfaced it. WHAT THAT CHANGES FOR THIS ITEM: (a) acceptance-criterion (1) 'root-cause the correct layer' now has a PRECEDENT rather than an open design question - the consumer layer, via a split_tsv-reader that does not use tab-as-IFS; adopting sibling's existing function is preferable to inventing a second dialect (11.4.251 - two answers to one question across two files that read the SAME scope rows is exactly the divergence that produced the round-1 'two readers of one scope file disagree' finding); (b) this is a RECURRENCE of a class already diagnosed and closed once in this codebase, not a novel discovery - the fix landed in one reader 2026-08-21 and the sibling reader kept the collapsing form, which is the 11.4.238 escape shape at the code layer: the lesson was captured in a comment instead of in a check; (c) the 11.4.238 coverage-escape note above is SHARPENED - the sibling's own comment already states that a fixture-scope suite cannot see the class, so the required automated check must exercise a row with a genuinely empty middle

ATM ID	Type	Status	Severity	Description
				field, not merely a well-formed one. BOB- Bug Queued Critical MEASURED LIVE 2026-08-26 against the running stack from LA 192.168.1.90 (NOT loopback - qBittorrent's localhost bypass would have exercised a different code path). Evidence: docs/qa/BOB-1 runtime_auth_verification_20260826.md. VERDICT: unauthenticated mutating LAN requests ARE currently accepted. Two live services, four named routes: 1. POST /api/v2/ torrents/stop port 7186 -> HTTP 200 with NO credentials (real torrent control) 2. POST /api/ hooks port 7187 -> 422 field-validation (requ REACHED FastAPI body validation, which run AFTER middleware) 3. POST /api/v1/schedule port 7187 -> 422 (same) 4. DELETE /api/v1/hoc <id> port 7187 -> 404 'Hook not found' (the handler EXECUTED A LOOKUP) The 422/404 responses are the decisive evidence, not the 2 a 401 never appears anywhere, and reaching body validation or a handler lookup proves n auth middleware intervened. THE ARMED TOKEN IS INERT. BOBA_API_TOKEN was arm in .env earlier this session on an operator decision. Port 7187 returns BYTE-IDENTICAL responses with and without it across three header shapes. Port 7189 does not open for it five header shapes. Root cause identified: doc compose.yml injects BOBA_API_TOKEN into EXACTLY ONE service - download-proxy (line 237) - and into neither qbittorrent-proxy-go r boba-jackett. And even on 7186, where it IS injected, POST /api/v2/torrents/stop returned unauthenticated, so injection alone is not enforcement. PER-SERVICE POSTURE (measured): 7185 qbittorrent-nox LIVE, binds same mutating surface visible from LAN as v the proxy 7186 download-proxy LIVE, binds 0.0.0.0 - mutating control accepted unauthenticated 7187 merge service LIVE, bi 0.0.0.0 - 34 routes via openapi.json, NO auth l at all 7188 bridge NOT BOUND - honest SKIP ' boba-jackett Go LIVE, binds * - writes 401-gat

ATM ID	Type	Status	Severity	Description
				reads OPEN incl. /api/v1/jackett/credentials 9 jackett LIVE, binds * EVERY live service binds interfaces. None is loopback-only. RELATION TO BOB-198 (its named subject was NOT running): qbittorrent-proxy-go is profiles:-ga (opt-in --profile go) and absent from the container set, so its '22 unauthenticated route neither confirmed nor refuted - honest 11.4.3 SKIP. The exposure recorded HERE is a DIFFERENT and ADDITIONAL surface on the Python service and the proxy. Do not treat th item as closing BOB-198. 11.4.238 COVERAGE ESCAPE - first class, equal standing to the bug the automated QA regime did NOT find this. 7 static gate check_cm_lan_routes_authenticate went through SIX independent review round proving routes are statically wired to auth middleware, and its own honest boundary (tracked BOB-197) states it asserts static wiring ONLY. Nobody had measured runtime until a directed agent did. This is precisely the 11.4 covenant's founding failure shape: a green ga over a broken-for-the-user reality. The owed remedy is an automated runtime check that would have caught it, with a RED capturing t exact exposure - not merely a fix to the route ANTI-BLUFF PROVENANCE: control needle - Jackett returned a real 401 through the SAME instrument, so the 200s are not instrument blindness; port 7188 returned exit-7, so absen are real. Negative control - a deliberately wro token still got 401 on 7189. Decisive probes 3/ deterministic. Mutating probes used non-exis ids against a provably EMPTY torrent list ([] before and after), so effect was nil and verific on both sides. Token referenced by name onl never printed (11.4.10); the report was leak- scanned with a needle proving the scanner fi on a known leak. ACCEPTANCE: (1) operator decision (11.4.66) on the intended posture pe service - loopback-only bind, real auth middleware, or accepted-LAN-risk-with- rationale; (2) whichever is chosen, an automa

ATM ID	Type	Status	Severity	Description
				runtime check per 11.4.238 that fails RED again today's exposure; (3) 11.4.108 layer-3/4 evidence on a clean deployment, not static analysis; (4) paired 1.1 mutation. === RELATED MEASUREMENT 2026-08-26 — A THIRD LAN-BOUND PORT (§11.4.6, recorded not merged) This item covers unauthenticated mutating routes on 7186 and 7189. A separate triage stream measured, and the conductor verified that port 7188 is ALSO LAN-bound: webui-bridge.py:447 binds ThreadingHTTPServer(("BRIDGE_PORT")) — the empty host string is INADDR_ANY, so it listens on ALL INTERFACES rather than loopback. NOT merged into this item per §11.4.214 (distinct-but-similar): this is a BIND-SCOPE fact about a different service, not demonstrated unauthenticated mutating routes on 7188. Whether 7188 exposes mutating routes without authentication is UNVERIFIED and is an open question this note raises. Recorded here the 7188 surface is not overlooked when this item's scope is settled — and so nobody later "discovers" it as new. BOB-204 Bug Qu critical WHAT: internal/jackettapi/credentials.go:234-253 (boba-jackett, port 7188) The DELETE credential handler checks the DELETE, then discards the errors of BOTH ` = envfile.Delete(...)` (line 240) and ` = d.Jackett.DeleteIndexer(id)` (line 249), then returns an UNCONDITIONAL 204 No Content. If the .env delete fails, the plaintext credential variables REMAIN ON DISK while the API reports the credential deleted. Second instance, same file: credentials.go:166-176 returns the error `env_write_failed_db_rolled_back` while the rollback's OWN error is discarded (`_ =` at :166 and :171) — the response body asserts a rollback that was never confirmed, a §11.4 bluff in the contract itself. SCOPE: 4 combined dangerous capabilities on one path (credential access + filesystem mutation + external side effect + irreversible delete) where §11.4.252 fail-close on-dangerous-combination requires only 2.

ATM ID	Type	Status	Severity	Description
				Composes §11.4.10 (credentials must never leak — a credential the operator believes deleted, on disk, IS the leak class) and §11.4.252. REPRODUCTION: read the cited lines; the disc is syntactic and unconditional. A runtime rep drives DELETE with the .env path made unwritable (chmod 0444 or a directory-level deny) and observes 204 while the variable persists in .env. WHY IT WAS NOT CAUGHT: t CM-DANGEROUS-COMBINATION-FAIL-CLOSE gate (constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh) enumerates .go at line 243 but gates its ast analyser on *.py at line 537, so Go falls to two matchers that key on `catch` — a keyword Go does not have. All 106 .go files are structurally unanalysable while being counted as analysed. That blindness is BOB-191; this item is the DEFECT it hid. ACCEPTANCE: (1) both discarded sites either handle the error or the handler returns a non-2xx naming the unresolved precondition per §11.4.252(2); (2) the env_write_failed_db_rolled_back code is only emitted when the rollback actually succeeded else a distinct honest code; (3) a RED test drives the unwritable-.env path and observes the pre-fix 204, flips GREEN post-fix (§11.4.115); (4) a golden-FALSE fixture proves the new guard does not refuse the healthy path (§11.4.201(1)). DISCOVERY CHANNEL (§11.4.238): found by a agent reading source during the BOB-191 investigation, NOT by the automated QA regime — this is itself a coverage escape and BOB-191 carries the escape audit. === VERIFICATION COMPLETE 2026-08-26 — BOTH DEFECTS CONFIRMED, RED AUTHORED AND WATCHED. FAIL === Line numbers re-derived, NO drift from the filing: :234-237 DB delete checked (500 on error) · :240-244 envfile.Delete error DISCARDED · :249 DeleteIndexer error DISCARDED · :253 unconditional 204. NOT AN OVERSIGHT — THE GODOC BLESSES IT. :210-211 reads: ".env and Jackett deletes are best-effort ...; failures are

ATM ID	Type	Status	Severity	Description
				silently swallowed. The 204 status reflects DB success." Any fix MUST delete that sentence or will read as authorizing the defect (§11.4.120 the doc asserting the old behaviour is part of what must be reconciled). MEASURED END STATE when the .env write fails: DB row GON plaintext RUTRACKER_USERNAME/ RUTRACKER_PASSWORD STILL ON DISK, API says 204. WORSE THAN THIS ITEM ORIGINAL FRAMED IT: the ENCRYPTED canonical copy destroyed while the PLAINTEXT copy survives UNMANAGED — the system's own record of the credential is gone, so nothing will ever clean up. DEFECT (2) IS WORSE THAN FILED — CHECKING THE ERROR WOULD NOT FIX THE LIE. The rollback calls Repo.Upsert(..., ifSet(prior.Username), ...); ifSet("") returns nil repos/credentials.go Upsert maps nil to COALESCE(excluded.username_enc, username_enc) = LEAVE UNCHANGED. So when the prior row had an empty field and the failed request set one, the compensating Upsert returns nil — it SUCCEEDS — and leaves the new value in place. The DB is not rolled back, there is NO error to check, and the API asserts a rollback anyway. Verified live by the third RED case. This means defect (2) is NOT one-line-ish: it needs the repository surface to express "set this field back to NULL" (delete-and-reinsert the prior row, or a tri-state in Upsert). Scope it separately from defect (1). THE RED: qBitTorrent-go/internal/jacketapi/bob204_failclosed_test.go (297 lines, NEW file gofmt clean, deliberately isolated from credentials_test.go so it cannot widen the T042/T042 diff). Failure injection is REAL, not mocked. envfile.mutate writes through <path>.tmp, so test pre-creates that path as a DIRECTORY -> os.OpenFile returns EISDIR (errno 21 probed envfile.Delete fails WHILE .env stays byte-identical with the credential in it. Chosen over chmod deliberately: opening a directory for writing fails for root too, whereas a chmod mechanism would silently no-op under uid 0

ATM ID	Type	Status	Severity	Description
				turn the RED into a FALSE GREEN. A CONTROL NEEDLE inside breakEnvWrites calls the real envfile.Delete and t.Fatal's if it unexpectedly succeeds — a blind instrument cannot produce conclusion. Measured: 3 FAIL (all three defects cases) + 1 PASS (the §11.4.201(1) golden-FALSE proving the REDs are not tautological and that the fix must not refuse the healthy path). Package-wide exactly 3 failures, all this stream zero skips. §11.4.10 honoured: no assertion or message prints a credential VALUE — names booleans only, fixtures are fake-not-a-real-*. HONEST BOUNDARY (§11.4.6): the GREEN has the flip is OWED, NOT CLAIMED. The stream forbidden from editing credentials.go, so RED was watched, GREEN was not. THE FIX, SCOPED (not applied): defect (1) IS one-line-ish twice - replace the two ` _ = ` with checked errors returning 5xx naming the unresolved precondition (env_delete_failed, jackson_cascade_delete_failed) and DELETE the :210-211 GoDoc sentence. Defect (2) is a repository surface change, separate scope. ADJACENT FINDINGS: (a) credentials.go is strictly WEAKER than its own sibling — indexers.go:207-209 handles the identical DeleteIndexer error and at least LOGS it; credentials.go:249 does not even log. Same package, same call, two standards. Repo is a concrete *repos.Credentials, not an interface, so a rollback-FAILURE path cannot be driven deterministically — relevant to §11.4.201. §11.4.241 if the fix wants that path testable. (b) The .gitignore false-null that nearly ate this deliverable is filed separately. SPOT-CHECKS the discards this item called benign — BOTH CONFIRMED BENIGN: client.go:140,153 ` _ = resp.Body.Close()` closes a body already being discarded before a retry; runs.go:112 ` _ = w.Write(...)` is post-WriteHeader, same class as credentials.go:274. Bonus: runs.go:43 ` _ = json.Unmarshal(...)` is benign but mildly lossy a corrupt ErrorsJSON silently reports error_count=0, documented as deliberate

ATM ID	Type	Status	Severity	Description
				degradation in its GoDoc. Cosmetic under-reporting is not an integrity issue; recorded so it is not re-investigated. BOB-207 Bug Queued Not critical WHAT: scripts/lib/ownership.sh:299 probe_location() reads stat -c '%u' and compares against ownership_operator_uid (id -u). It NEVER reads '%g'. Meanwhile ownership_operator_gid is defined at scripts/lib/ownership.sh:50 and consumed ONLY by scripts/ownership_repair.sh:360 — so the REPAIR establishes uid AND gid, while the PRECONDITION that verifies the repair work checks uid ONLY. The precondition can therefore report ok while exactly half the property the repair exists to establish is wrong. MEASURE: LIVE ON THIS HOST, UNPRIVILEGED (a setgid directory is all it takes — no loopback, no mount, no sudo): dir: uid=1000 gid=10 mode=2755 (chmod wheel + chmod g+s, both unprivileged) real file created there: uid=1000 gid=10 probe_location verdict = ok rc=0 <-- operator gid is 1000, file is 10 SEVERITY RATIONALE: this is the SAME CLASS that BOB-206 alleged (a probe reporting ok while ownership is only partly held) but on an axis that is LIVE rather than hypothetical, needs no unusual filesystem, and reproduces with the unprivileged commands. BOB-206 was dismissed because this is the real defect that investigation surfaced underneath it. ACCEPTANCE: (1) probe_location reads and compares gid as well as uid, OR the asymmetry is deliberately justified in-source with the reason (if only uid matters to the user's visible goal, say so and explain why the repair sets gid at all); (2) a RED fixture builds the setgid directory above, observes ok pre-fix, flips to a refusal post-fix (§11.4.115); (3) a golden-FALSE fixture proves a correct uid+gid location is still ok (§11.4.201(1)); (4) whichever way it is resolved, precondition and repair agree on WHICH property they are talking about — the disagreement is the primitive defect (§11.4.238). DISCOVERY CHANNEL (§11.4.238): found by the BOB-206 verification stream, not by the

ATM ID	Type	Status	Severity	Description
				automated QA regime. No existing test exercising a gid mismatch. === CONFIRMED 2026-08-26, AUTHORED — AND THE FIX DIRECTION IS THE OPPOSITE OF WHAT THIS ITEM ASSUMED === CONFIRMATION (re-derived against the dirty tree; line numbers happen to match): <pre>probe_location() at scripts/lib/ownership.sh:299-320 reads stat '%u' twice (:file branch, :314 dir branch), compares again want at :301, and NEVER reads '%g'.ownership_operator_gid is defined at :50 and EXACTLY ONE consumer —ownership_repair.sh:360 — verified across tracked AND untracked files, needle-proven (needle ownership_operator_uid -> 13 files, negative control -> 0; untracked sweep needle 8 files, ownership_operator_gid -> 0); every o hit is a docs/DB carrier or comment. THE RE SETS GID AT TWO SITES, not one: :719 chown "\${OP_UID}:\${OP_GID}" WRITES both, and :9 find_args selects on (! -uid OR ! -gid). FALSE REPRODUCED unprivileged (chgrp to a group from id -G + chmod g+s): dir uid=1000 gid=10 mode=2755, a REAL file created inside lands uid=1000 gid=10, probe_location verdict = ok rc=0. Needle arm: an absent path returns 'abs rc=1 — the probe CAN refuse, so the ok is real DESIGN QUESTION ANSWERED: **NARROW REPAIR. gid is NOT load-bearing.** This item filed assuming the PROBE was deficient.</pre> Measured, that is backwards. - With correct uid and gid=10: edit, read, rename, move, chmod, utime, delete are ALL PERMITTED — including recovery from chmod 000 with NO elevation. Owner-class bits are selected by uid match and chmod is owner-only, so the operator can always restore access. - Every FR-001 / FR-002 / FR-003 / FR-010b is phrased as "owned by the account NO requirement mentions gid. - DECISIVE, conductor-verified in spec.md: the Constraint section blesses "Group-based workarounds (shared groups, permissive modes, access-control lists) are acceptable implementation routes",

ATM ID	Type	Status	Severity	Description
				Dependencies makes "shared group plus inherited permissions" NECESSARY if the container platform cannot map the in-container account to the operator's identity. => WIDEN THE PROBE would make FR-010 REFUSE TO START on a configuration the spec explicitly sanctions — a §11.4.201(1) false-positive refusal the exact failure class this project keeps finding in its own gates. => AND THE REPAIR IS ALREADY DOING HARM: measured, after repair a new file inherits gid 1000 instead of 10 with setgid bit still set and NO diagnostic. The repair silently dismantles a spec-sanctioned shared-group setup. THE RED: tests/unit/test_ownership_gid_agreement.sh (NEW file, 4 lines, bash -n clean). It deliberately does NOT assert "the probe must call stat %g" — that is an implementation assertion that survives a cosmetic fix AND hard-wires one fix direction asserts the user-observable invariant: probe_location(L) == ok IFF ownership_repair --dry-run selects ZERO items under L. Measurement 4 passed / 1 failed / EXIT=1, the failure being PRECONDITION certifies it (ok) while the RED reports it broken (selects:2)". VALIDATED AGAINST BOTH CANDIDATE FIXES from patches COPIES (production untouched): baseline FAILURE · narrow-the-repair PASS(0) · widen-the-probe PASS(0). Golden-FALSE passes in all three, so neither direction can go green by refusing everything. That is what makes this RED fix-direction-agnostic rather than a vote. THE FIX NOT APPLIED — scripts/ownership_repair.sh - find_args+=(\(! -uid "\${OP_UID}" -o ! -gid "\${OP_GID}" \) -printf '%U\t%G\t%m\t%p\0') + find_args+=(! -uid "\${OP_UID}" -printf '%U\t%G\t%m\t%p\0') BOB-208 WHAT: scripts/ownership_precondition.sh:342-348 documented the docker branch of detect_rootless() as documented-not-measured (accurate — verified verbatim). The header then claims the branch
BOB-210	Bug	Queued	major	

ATM ID	Type	Status	Severity	Description
				built so that ‘an unverified reading can never manufacture a refusal’. IT CAN. Line :413 em CONFIDENT rootful verdict whenever docker info succeeds with non-empty output containing no name=rootless field. Only the FAILURE mode fall through to unknown (:387-394). THE PATENT TO HARM: PUID=0 is declared in docker-compose.yml:32 (and again for jackett — deliberate per project policy). With a confident rootful verdict, that reaches the R3 refusal at :517. So a genuinely ROOTLESS Docker host whose docker info output SHAPE differs from documented one gets a FALSE REFUSAL produced by a branch the source itself admitted was never measured — precisely the outcome the header promises is impossible. THE ORACLE PROBLEM (§11.4.245 independence): the shim tests DO cover this path (tests/unit/test_ownership_rootless_detection.sh:283) — the shim’s output shape was authored from the SAME documentation as the code. Oracle and code share the unvalidated premise, so the test cannot discover that the premise is wrong. It confirms the code matches the doc; nobody has confirmed the doc matches Docker. ACCEPTABLE (1) either the header claim is corrected to match :413’s real behaviour, or :413 is changed to yield unknown when the reading is unverified — the two must agree (§11.4.6); (2) the docker branch premise is validated against REAL docker info output from a real rootless daemon, or the branch is honestly marked unmeasured at the point of USE, not only in the header; (3) if validation is operator-gated (needs a rootless Docker host), record it as such per §11.4.21 rather than leaving the header’s promise standing. ALSO VERIFIED GOOD, recorded so it is not re-investigated: the unknown branch is handled IDENTICALLY at both consumers (R3 :519-531 :832-836 — both a named SKIP, both return 0, neither refuses), ROOTLESS_VERDICT is cached at :487 so both read ONE measurement, and the carrier-trap golden-FALSE fixture EXISTS

ATM ID	Type	Status	Severity	Description
BOB-211	Bug	Queued	minor	(test_ownership_rootless_detection.sh:279-30 drives name=rootless-lookalike inside a profile path and must NOT read rootless; structural guard at :396-411 uses exact field equality, not substring). DISCOVERY CHANNEL (§11.4.238) found by the BOB-206 verification stream. STATUS: LATENT on this host (measured: all s declared locations sit on ONE btrfs mount, full ownership-expressive) and OPERATOR-GATED for verification. Retained from the dismissed BOB-206 rather than discarded with it, because the scenario is well-founded elsewhere: the declared mount is a udisks auto-mounted REMOVABLE NVMe, so another operator's portable drive being exFAT or NTFS is entirely ordinary. TWO RESIDUALS: (1) REPAIR-LAYER DIAGNOSIS. On a uid-flattening mount whose flattened uid is NOT the operator, probe_location correctly REFUSES (verified: uid=0 and uid=100999 both refuse). But it then hands ownership_repair.sh a chown that will return EPERM, and nothing in the repair path is fstyp aware — so the operator sees a permission failure whose real remedy is a REMOUNT OPTION, which no message says. Reasoned, not proven: the shim used to verify BOB-206 mode the stat READ, not the chown WRITE, and cannot settle whether chown actually fails. (2) MODE CONTRACT. vfat flattens MODE via fmask/dm so the preserve_mode: true / 600 contract on and config/boba.db would be silently unenforceable there. §11.4.10-adjacent: a credential file believed to be 600 that is world-readable by mount option is a leak the mode check cannot see. WHY OPERATOR-GATED: a genuine end-to-end RED needs a real uid-flattening mount. The privilege-free route is bindfs -u (FUSE; fusermount IS present and user_allow_other IS set) — but bindfs is ABSENT on this host, as are mkfs.vfat and losetup. So verification needs either a bindfs install or a privileged loopback mount. Neither was

ATM ID	Type	Status	Severity	Description
				attempted (§11.4.21 — high-blast-radius host mutation is operator-gated). ALSO RECORDED NO test anywhere exercises a uid-flattening filesystem — vfat/exfat/ntfs/flatten/loopback/mkfs/losetup/bindfs/fusermount all measure across the three ownership test files, needle-proven (probe_location = 3 hits through the s instrument). ACCEPTANCE: (1) operator decide whether to install bindfs or provide a privilege loopback so the RED can be built; (2) if verified the repair path gains fstype-aware diagnosis naming the remount remedy; (3) the mode contract either detects mode-flattening mount and refuses, or documents honestly that it can be enforced there. DISCOVERY CHANNEL (§11.4.238): retained residual from the BOB-2 verification stream. WHAT: .gitignore:31 is a deny-all glob <i>credentials</i> followed by a HAND-MAINTAINED per-file allowlist of ! exceptions (lines 34-50). Any NEW source file whose name contains ‘credentials’ (‘creds’) is silently ignored: git add refuses it and git status shows NOTHING. The author gets no signal at all — the file simply does not exist as the commit path is concerned. HOW IT SURFACED: the BOB-204 stream authored a R test named <i>credentials_failclosed_test.go</i>. It vanished. The agent noticed only because it was looking for the file it had just written. It renamed to <i>bob204_failclosed_test.go</i> to escape both <i>credentials</i> and <i>creds</i>, and the file became visible. A silent deliverable loss, caught by luck rather than by any gate. INDEPENDENTLY RE-CONFIRMED BY THE CONDUCTOR, control-needle proven: git check-ignore -v --no-index qBitTorrent-go/internal/jackettapi/credentials_failclosed_test.go -> .gitignore:31:<i>credentials</i> (IGNORED) git check-ignore -v --no-index qBitTorrent-go/internal/jackettapi/zzz_probe_test.go -> no match (NOT ignored) <– the needle: instrument proven seeing, so the hit above is real git check-ignore
BOB-212	Bug	Queued	critical	

ATM ID	Type	Status	Severity	Description
				–no-index frontend/e2e/credentials.spec.ts -> .gitignore:31:credentials (IGNORED) git ls-f –error-unmatch frontend/e2e/credentials.spe -> tracked SECOND, LATENT INSTANCE: front e2e/credentials.spec.ts is matched by the sam rule and survives ONLY because it is already tracked (git honours the index over .gitignore tracked paths). Delete-and-re-add it — an ordinary refactor, a branch operation, a file r — and it disappears silently. It is one routine operation away from being lost. (For contrast credential-edit-dialog.component.spec.ts is genuinely safe: it is covered by the DIRECTOR rule !frontend/src/app/jackett/credentials/ at 43, not by a per-file entry.) WHY THIS IS A §11.4.201(6) FALSE-NULL, NOT MERELY AN INCONVENIENCE: the blind instrument and t clean tree return the identical quiet zero. git status showing nothing means BOTH ‘no new files’ AND ‘a new file exists but is invisible’. T is no signal that distinguishes them. The com path — the thing every other gate’s output eventually has to travel through — cannot se own blindness. WHY IT IS NOT SIMPLY ‘ADD ANOTHER ! LINE’: that is the mechanism tha produced the defect. A hand-maintained allo against a deny-all glob has the §11.4.251/ §11.4.205 shape — every future legitimate credential-named source file needs a hand ec nobody will remember to make, and the failu mode of forgetting is SILENT. The same shape BOB-205’s hand-maintained DANGER_ROOTS list that must be edited in lockstep with the tr with no guard that notices when it was not. TENSION TO RESOLVE HONESTLY (§11.4.10 v §11.4.201(1)): the glob exists for a real reason §11.4.10 forbids credential material reaching and a broad deny-all is the conservative-safe default. Narrowing it carelessly would re-ope genuine leak channel. So the fix is NOT ‘delet the glob’. Candidate directions, operator deci (§11.4.66): (a) narrow the deny to what actua carries secrets — extensions and directories

ATM ID	Type	Status	Severity	Description
				(.env, secrets/, credentials.json yaml yml txt ... enc) rather than every path containing the word "secret"; (b) keep the deny-all but add a GATE that FAILS when an untracked file matches an ignore rule AND lives under a first-party source root AND has a source extension, so the swallow becomes loud instead of silent; (c) both. (b) alone closes the file to null even if the glob stays exactly as it is, and it is a smaller change. ACCEPTANCE: (1) a NEW credential-named source file under a first-party source root either commits normally or produces a LOUD refusal naming the rule that blocked it and the remedy — never silence; (2) a RED that creates such a file and asserts the current silence, flipping to the loud path post-fix (§11.4.115); (3) golden-FALSE proving real secret material (a .env file a key) is STILL ignored — the §11.4.201(1) guarantee because a fix that leaks credentials to close a file to null is strictly worse than the defect; (4) frontend/e2e/credentials.spec.ts is made safe by rule rather than by the accident of already being tracked. DISCOVERY CHANNEL (§11.4.238): found by the BOB-204 stream when its own deliverable disappeared — NOT by the automated QA regime and not by any gate. Nothing in the repo checks that an authored file actually became visible to Coverage-escape audit: no surface exists for the class at all. === SCOPED AND MEASURED 2026-08-26 — RECOMMENDATION (b), AND (c) DISPROVEN NOT MERELY DISFAVOURED === TASK 1a — TRACKED FILES SURVIVING ONLY BEING TRACKED: 3, not 157. A raw sweep of 2795 tracked paths hit 157 ignore rules — but of those are NEGATIONS (! lines), i.e. explicitly re-included and safe BY RULE. Reporting 157 as the blast radius would have been a §11.4.201(1) false positive; the investigating stream caught itself. Two independent instruments (check-ignore -v filtered on non-negated patterns, and bare check-ignore which emits only truly-excluded paths) produced IDENTICAL sets. 1. frontend/e2e/credentials.spec.ts — .gitignore:31 credentials HAZARD (the known one, 109-line Playwright

ATM ID	Type	Status	Severity	Description
				spec) 2. tests/security/ test_jackett_autoconfig_secrets.py — .gitignore:260 — HAZARD, NOT NAMED IN THIS ITEM WHEN FILED. Conductor-verified: matched 1:51, currently tracked. It is a 113-line SECURITY PEN TEST asserting no credential value reaches an HTTP body, traceback, or log. Losing it silently would remove the test that guards §11.4.10 — deny-all glob's own purpose. That is the sharp instance available: the rule protecting credentials is one refactor away from deleting the test that proves credentials are protected submodules/jackett — .gitignore:260 — NOT a hazard: mode 160000 gitlink, and that entry is documented as deliberately inert for git (it st CodeGraph per §11.4.79). THE MECHANIC TH DECIDES THE FIX: the directory negation at : re-includes the DIRECTORY ONLY. A new frontend/src/app/jackett/credentials/credentials.guard.ts is STILL swallowed by :3 because <i>credentials</i> contains no slash and is therefore BASENAME-scoped — the allowlist directory buys nothing for new files. Of the 1 safe files, 124 are held by ONE robust glob negation (!docs/qa//.log) and 13 by fragile h maintained per-file entries (all 13 current live, none stale — but each is a future silen loss). TASK 1b — ALREADY-SWALLOWED UNTRACKED FILES: 5 of 60,217 ignored+untracked paths. Two are real lo (the tracker-credentials export twins and BOB-124 .txt evidence — filed as their own item); set_password.py is recorded UNKNOWN: provenance and deliberately called lost work; download-proxy/qbittorrent_creds.json is the POSITIVE CONTROL — shape-inspected keys-only (ne values) as [password, username], genuine credential-bearing, CORRECTLY ignored. TASK 2 — MEASURED IN A SCRATCH TREE, source paths x 15 secret paths: baseline (f 316-line file) swallows 10 leaks 0 (a) EXACTLY AS THIS ITEM WORDED IT swall

ATM ID	Type	Status	Severity	Description
				0 leaks 3 (a) repaired to plug those 3 swallows 1 leaks 1 (b) deny-all unchanged loud gate swallows 0 leaks 0 (a) LEAKS download-proxy/qbittorrent_creds.json — REAL CREDENTIAL FILE SITTING IN THE T TODAY — because credentials.json does not match “creds”. It also leaks my_password and service_credentials. Repairing those s leaves service_credentials: an extension-scoped deny cannot cover EXTENSIONLESS secret files without collapsing back into the deny-all it was meant to replace. So (a) as item worded it is DISPROVEN, not merely disfavoured — and I wrote (a) first in the candidate list, which was wrong. WHAT (NEWLY) ALLOWS INTO GIT: NOTHING — the set is EMPTY. No pattern changes, so the §11.4.10-vs-§11.4.201(1) proof obligation discharges trivially. No variant of (a) can make that argument. TASK 3 — THE RED: tests/security/ test_gitignore_swallow_is_loud.sh (filename verified not swallowed BEFORE writing — joke this item is about). It asserts the acceptance DISJUNCTION (commits normally OR refuses loudly), so it is FIX-DIRECTION AGNOSTIC and leaves the direction an operator decision. The load-bearing assertion is A3: git status is BYTE-IDENTICAL with a without the file — world A (authored) 330ceb52074cdbbe, world B (never made) 330ceb52074cdbbe. That is the false-null proven directly, not argued. Proven NOT a tautology: RED_MODE=1 -> exit 0; guard stand-in -> exit 0; narrowed glob -> exit 0; the §1.1 mutation DELETING the globs -> exit 0 with 11 leaks caught, which is what makes golden-FALSE load-bearing rather than decorative. Exit 2 is deliberately distinct from exit 1 so a blind harness can never read it as “defect present”. BEYOND WHAT THIS ITEM FRAMED: it is not a credentials/creds* problem. secrets, _password* and *_user* have

ATM ID	Type	Status	Severity	Description
				EACH already swallowed something real.** *_user* is the broadest — it will match any future *_user.txt / .json evidence or fixture. The must be evaluated against the whole rule set, not the one glob that surfaced first. === SECOND L TRAP HIT, SAME SESSION (2026-08-26) === TH BOB-207 stream chose the natural filename test/unit/test_ownership_user_owned_gid.sh for its RED. Conductor-verified: that path is SWALLOWED by .gitignore:60 _user. The stream checked before writing (having been warned) and renamed to test_ownership_gid_agreement.sh, which is clean. That is the SECOND deliverable one session that the deny-all globs would have silently eaten — the first (BOB-204's credentials_failclosed_test.go) was caught only after the fact. It also confirms this item's own "beyond the item" warning empirically: _user* the broadest of the four globs and it caught a whose name contains no secret and no credentials — merely the word 'user' inside 'user_owned' which is the FEATURE'S OWN NAME (spec 00 user-owned-downloads). Any test named after this feature is a candidate for silent loss. Two in one session, from two independent streams not a coincidence rate — it is the defect operating normally. WHAT: the §11.4.252 fail-closed gate's extension list omits shell entirely. Measured verbatim by the conductor at constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh:45 exts="\${DANGEROUS_COMBO_EXT:-py go rs c cpp h hpp java cs js ts jsx tsx php rb}" No sh. bash. THE CONSEQUENCE, MEASURED: script IS a declared DANGER_ROOT (scripts/pre_build_verification.sh:1511). It contains 71 tracked .sh files and 3 .py files. So 71 of 74 source files in a root the gate is explicitly pointed at SILENTLY INVISIBLE — while invariant 39's driver prints "no fail-open anti-pattern across first-party source root(s)". Project-wide there 198 tracked .sh files (negative control *.zzz = 0
BOB-213	Bug	Queued	critical	

ATM ID	Type	Status	Severity	Description
				instrument proven seeing). WHY THIS IS THE WORST OF THE THREE HOLES: BOB-205 is a scope hole (root never looked at). BOB-191 is a matcher hole (Go enumerated but unanalysed). THIS one is worse than either, because the root is declared, IS scanned, IS counted as covered, and 96% of what is in it was never examined. The count in the driver's own summary is an underestimate, count presented as a census. AND SHELL IS THE HIGHEST-RISK LANGUAGE HERE, not the low-level of the project's orchestration, credential handling, container control and gates are shell. start.sh alone is 1288 LOC and is, per CLAUDE.md, THE orchestrator. Fail-open in shell is also unusually easy to write — a bare <code>`cmd \\\  true`</code>, an unchecked <code>`\$?`</code>, <code>aset +eregion</code>, a swallowed <code>2dev/null-</code> and §11.4.67(6) already records a live instance of exactly this class (a bare <code>exec</code> redirection silencing an interactive shell for its lifetime). CORRECTION TO A PREVIOUSLY-STATED ACCEPTANCE CRITERION (§11.4.6): I asserted, when filing BOB-205 that "the honest-blindness path (§11.4.3 SKIP-with-reason, which the gate already implements correctly for unenumerated extensions) is preserved". THAT IS WRONG. The gate's honest SKIP-with-reason exists for the PYTHON ARM degradations, NOT for extensions absent from the ext list. An unenumerated extension is SILENT false-null, not an honest skip. The distinction is load-bearing and I stated it backwards. NOTE ON LINE NUMBERS: the BOB-205 stream cited the ext list at :318; the conductor measured it at :457. Both are correct at their read times — a sibling stream (BOB-195 r7) is actively editing this file. Re-derive before acting. ACCEPTANCE: (1) either shell is added to the ext list WITH an arm that can actually analyse it, or unenumerated extensions produce an explicit UNANALYSED verdict per file — never a silent pass (this is the same analyser-registry for BOB-191 needs, and doing it once covers both

ATM ID	Type	Status	Severity	Description
				(2) the driver's summary reports files ANALYSED, not files present, so an under-c cannot masquerade as a census; (3) a RED planting a shell fail-open under scripts/ asserting it is SEEN; (4) golden-FALSE per §11.4.201(1) – a correctly fail-CLOSED she guard must NOT be flagged; §11.4.67's own brace-scoped exec form is the natural fixture. COMPOSES: BOB-191 (matcher hole same primitive: classify by local shape, r consult semantic role; and the analyser- registry fix closes both) and BOB-205 (scd hole). Per §11.4.250 these are three sympt of one primitive defect; per §11.4.214 the stay three items because the fixes differ. DISCOVERY CHANNEL (§11.4.238): found by th BOB-205 verification stream, confirmed independently by the conductor. Not by the automated QA regime. BOB-214 Bug Queued critical WHAT: DANGER_ROOTS = (download-proxy/src plugins scripts qBitTorrent-go frontend/src) at scripts/ pre_build_verification.sh:1511. The REPOSI ROOT itself is not among them (conductor- verified: no "." entry). Everything living the top level is scanned by no arm. THE L HIT: webui-bridge.py – tracked at root (conductor-verified), 466 LOC, a live HTTP service on port 7188 (BaseHTTPRequestHandler:85, do_GET/ do_POST:92-97, request path read at :103, outbound urlopen at :273, environment credentials read at :55-74). The gate REPO A REAL HIT AT :295 when pointed at it directly. This is not a hypothetical gap: existing gate, unmodified, finds a genuine defect in this file the moment scope reach it. ALSO UNSCANNED AT ROOT: 13 first-part shell scripts including start.sh (1288 LOC per CLAUDE.md the project's orchestrator a the sole sanctioned container-control entr point), stop.sh, ci.sh, install-plugin.sh, the credential-handling init-qbit-

ATM ID	Type	Status	Severity	Description
				password.sh / fix-qbit-password.sh. (These additionally invisible for the separate LANGUAGE-hole reason – shell is not in the gate's ext list – so root files get missed twice over, by scope AND by extension.) SCALE THIS SITS IN: 266 files in scope / 5 out of scope – 66% of the gate-visible first party corpus is never scanned. Other unscanned roots measured: tests/ (324 files, excluded by-intent but UNDECLARED – an undeclared exclusion is exactly what §11.4.224(E) fences against), extension/ (108, shipped browser extension), docs/ (51), challenges/ (18), tools/ (1, and it CONCEALS 3 REAL HITS at plugin_update_automation.py:189,198,215), frontend/e2e + 2 configs (5). SO THE SCOPE HOLE CONCEALS AT LEAST 4 REAL HITS the gate itself finds when pointed at them (1 in webui-bridge.py, 3 in tools/). Invariant 39's reported count is an under-count, not a census. ACCEPTANCE: (1) repository root and tools/ are scanned, or explicitly fenced with a stated reason per §11.4.224(E) – silence is not an exclusion; (2) the 4 concealed hits are triaged (each is either a real defect to fix or a false positive to fix in the gate – both are findings); (3) tests/ (324 files) gets OPERATOR decision per §11.4.66 – production only is defensible but must be DECLARED; (4) RED proving a root-level fail-open is seen (5) golden-FALSE proving genuine build artefacts and vendored trees stay excluded. FIX DIRECTION (measured by the BOB-205 strategy not guessed): a §11.4.251 manifest is feasible but the source of truth must be chosen carefully – docker-compose.yml build context, MISS plugins/, frontend/src, cmd/boba-ctl, webui-bridge.py; language markers (go.mod/package.json) MISS plugins/ and webui-bridge.py. Only derive-from-git-tracked-source-extensions minus a declared §11.4.224(E) exclusion fence reaches every gap. Note the derivation must be built either

ATM ID	Type	Status	Severity	Description
				way – if the operator prefers keeping a ha list, the omission-guard that audits it is SAME computation; the only question is whe it drives the scan or audits the list. DISCOVERY CHANNEL (§11.4.238): found by th BOB-205 verification stream while enumerat the full gap list – the item it was verify named only one missing root. Not by the automated QA regime. === CORRECTION 2026-08-26 – THE "4 CONCEALED REAL HITS" C IN THIS ITEM IS REFUTED (§11.4.6) === Whe filed this item I wrote that the scope hol "conceals at least 4 real hits the gate it finds when pointed at them". A dedicated triage stream ran the UNMODIFIED gate again scratch copies, reproduced all four hits exactly, and triaged them: **4 of 4 are FA POSITIVES.** The instrument was control- needle-proven in BOTH directions first (fi on a plantedexcept:pass, stayed silent on clean log-and-reraise), so this is evidenc not a null. All four are the SAME detecto class – shape (A2) "silent default return" which is precisely the BOB-189 false-posit shape: the returned literal IS the functio designed refusal value, consumed by a call that checks it. - webui-bridge.py:295 (_is_root_liveness_probe): False -> send_error(502) at :438. The except DENIES lenient 200 path. Fail-CLOSED. Combines ZE of the six §11.4.252 capabilities – a pure classifier over self.path/self.command. - tools/...:189 (_download_url): None -> :11 handles it explicitly (warn + continue), r write. Handler BOUNDED to (URLError, HTTPError); anything else propagates. - tools/...:198 (_extract_version): "unknown gates nothing – the update decision is the HASH compare at :116. (It IS a bareexcept:though, so Ctrl-C is swallowed separate signal-handling defect, filed wit the README item.) - tools/...:215 (_validate_plugin): False -> :151 returns

ATM ID	Type	Status	Severity	Description
				BEFORE the write at :165. This IS the refu Canonical BOB-189 shape. WHAT THIS DOES A DOES NOT CHANGE. The scope hole itself is UNCHANGED and still real – the repository, and tools/ are in no DANGER_ROOT, verified verbatim at scripts/ pre_build_verification.sh:1511. What chang is the ARGUMENT: I justified this item par on "it hides four known defects", and that justification is wrong. The correct justification is narrower and does not dep on any current hit: webui-bridge.py is a LAN-BOUND HTTP service (see below) handling request paths, outbound calls, and environ credentials – a surface that belongs in scope on its capabilities, whether or not it currently contains a defect. NEW FACT RAI THE STAKES – 7188 IS LAN-REACHABLE. webui-bridge.py:447 binds ThreadingHTTPServer(("BRIDGE_PORT)) – conductor-verified. The em host string is INADDR_ANY, so port 7188 listens on ALL INTERFACES, not loopback. Neither localhost-only nor internet-exposed (internet only if forwarded). No severity assigned to :295 because it is fail-closed but this is the argument for putting the r in scope: a live LAN-bound service should be scanned on principle, not after a defect is found in it. webui-bridge.py IS LIVE, DECISIVELY (§11.4.124 never engages – do NOT treat it as superseded): 142 tracked files reference it, including config/served_ports.yaml and the credential-leak-audit challenge; download-proxy actively probes it via bridge_health() at download-proxy/src/api/__init__.py:198; newest of 1 commits is a real functional fix (6e3da73, 2026-06-14). Critically, served_ports.yaml:66-69 records that the G webui-bridge is a SEPARATE binary the qbittorrent-proxy-go container never starts so NOTHING ELSE BINDS 7188 and this Python file is the only thing serving that port.

ATM ID	Type	Status	Severity	Description
				tools/ RECOMMENDATION (§11.4.224(E)): FENC OUT, DECLARED – exclusion class "non-shipp developer utility". Evidence: 2 tracked fi neither gitignored; the script is invoked NOTHING (control-needle-proven: 91 needle hits, 0 negative control – referenced only its own README, its own docstring, and a tracker note; zero hits across *.sh/*.yaml/ *.py/Makefile outside tools/); its README self-declares "not invoked by the normal start/stop flow"; last functional commit 2026-04-12. Because it is FIRST-PARTY, §11.4.224(E) requires the exclusion carry tracked §11.4.197 item – and the two REAL defects found in it must be tracked regard of scope, since a hand-run --update still executes them. BOB-215 Bug Queued major WHAT: cmd/boba-ctl/main.go:498-509 authMethod(). Its default branch (:507) returns remote.AuthSSHKey. A deploy-regist entry whose auth: key is misspelled, empty set to an unrecognised value therefore resolves SILENTLY to SSH-key authenticatio rather than refusing to act. WHY IT IS A §11.4.252 VIOLATION: this is a credential- selection decision on a path that also performs remote mutation and external side effects. §11.4.252 requires a path combin >=2 dangerous capabilities to FAIL CLOSED verify every precondition, refuse when any unverifiable, and name the unresolved precondition. Silently substituting a defa credential MECHANISM for an unreadable declaration is the textbook fail-open shap the operator's intent was not determined, the code proceeded anyway using a method t may not have chosen. WHY NO GATE CAUGHT I three independent reasons, each sufficient its own – (a) cmd/boba-ctl is in no DANGER_ROOT (BOB-205); (b) even in scope, files are structurally unanalysable by the current gate (BOB-191); (c) this shape is semantic default-branch decision, not one

ATM ID	Type	Status	Severity	Description
				the two text patterns the non-Python arms match. It was found by a human-equivalent read, which is precisely the §11.4.238 esc class. HONEST QUALIFICATION (§11.4.6): cm boba-ctl/main.go has NO_ = err, NO emptyif e nil {} , and NO silentreturn niltoday – grep- verified with a control needle against qBitTorrent-go/internal/config/config.go. finding is the exception, not one of many; scope hole around boba-ctl is otherwise LA (it hides nothing else live, it guarantees future one lands unseen). ACCEPTANCE: (1) unrecognised/empty auth: value REFUSES with error naming the offending key and its declared value, never silently defaults; (2) RED driving a typo'd auth: and asserting today's silent SSH-key resolution, flipping refusal post-fix; (3) golden-FALSE proving every LEGITIMATE auth: value still resolve correctly – a false refusal here would break real deploys (§11.4.201(1)). DISCOVERY CHANNEL (§11.4.238): found by the BOB-205 verification stream while confirming boba- qualifies as a §11.4.252 surface. BOB- Bug Queued major WHAT: three carrier classes fire in the gate's PRIMARY mode, not only its degraded text fallback. Measured ast_rc=1 AND text_rc=1 (both modes report hit): (a) a COMMENT or DOCSTRING quoting credential anti-pattern is reported as a L credential default – both spellings; (b) comment or a string constant holdingtry { x catch (e) { } is reported as an empty catch, once per carrier line. Shape (b), and the C-family of shape (a), are LANGUAGE-AGNOSTIC GREPS NO structural counterpart in any mode – so there is no parser to be immune. WHY IT MATTERS: the gate's header carried a blank claim that "A parser is immune to both by construction". That is OVER-BROAD and now corrected in place, scoped to the Python T With shapes where it is actually true.

ATM ID	Type	Status	Severity	Description
				Everything else – every non-Python extension and the credential/empty-catch text matches even on Python – has no AST arm at all, so immunity never applied there. A §11.4.201(false refusal in the PRIMARY mode is materially worse than one in a fallback mode expects to be exact: it refuses provably-healthy code on the path the gate is trusted on. THE CONTROL NEEDLE THAT SHARPENS IT (recorded so the next reader inherits the measurement): a #-COMMENTED import does NOT poison the licensing table (text=0), because those regexes are line-anchored. So the blindness is to STRINGS specifically, not non-code generally. That distinction is what makes the comment-strip fix tractable for class and not for the others. WHY IT WAS FIXED IN THE ROUND THAT FOUND IT (§11.4.6 honest boundary, and this is the right call closing these needs a PER-LANGUAGE comment-and-string model across every configured extension. That model's failure direction is the UNDER-reporting one – a mis-parsed string region silently swallows real violations for the remainder of the file. The same reason made the round DECLINE the triple-quote fence counter for OVER-1 while ACCEPTING the comment strip for OVER-A: the comment strip's ambiguity resolves toward KEEPING text (probably by three control needles – real violation comment, # inside a string, escaped quote before # – all 1/1), whereas a fence count ambiguity resolves toward DROPPING text. Direction of failure, not difficulty, is the discriminator. Building a full lexer here would import the under-reporting failure mode this gate refuses everywhere else. OPERATIONAL DECISION (§11.4.66 / §11.4.197): whether to build per-language comment-and-string model is a consumer call, not a default this gate should pick unilaterally. Options: (a) accept the over-reports and DISCLOSE them per class (what the round did – a new MODE-INDEPENDENT

ATM ID	Type	Status	Severity	Description
				CARRIERS section now enumerates all three) (b) build the models per language and own under-report risk with a fixture per langu (c) narrow the language-agnostic greps so only fire where a structural arm exists, trading coverage for precision. RELATION SIBLINGS (§11.4.214 distinct-but-similar, deliberately not merged): BOB-189 is the g flagging a fail-CLOSED SSRF guard – a semantic-role miss in the PYTHON arm. This a CARRIER miss (comment/string read as coo that is MODE-INDEPENDENT. BOB-213 is a language absent from the ext list entirely All three share the primitive §11.4.250 na in BOB-191: classify by local syntactic sh never consult semantic role – but the fixe differ, so the items stay separate. ACCEPTANCE: (1) the operator decision abov taken and recorded; (2) whichever path is chosen, each of the three classes has a fixture proving current behaviour, so a fu change cannot silently alter it; (3) if (b chosen, a golden-FALSE per language provin REAL violation adjacent to a carrier is st caught – the under-report guard, which is whole risk. DISCOVERY CHANNEL (§11.4.238) found by the BOB-195 round-7 remediation stream while closing a different finding, by the automated QA regime and NOT by the independent reviewer that audited the same file in round 6. BOB-217 Bug Queue critical WHAT: tools/ plugin_update_automation.py update_plugin(combines three §11.4.252 dangerous capabilities and is gated by nothing that could stop a hostile payload: - UNTRUSTED INPUT: 14 URLs across four GitHub repositories, THREE of which are third-par PERSONAL repos, not the official qbittorre organisation. - MUTATION: writes plugins/ <name>.py. - DEFERRED CODE EXECUTION: qBittorrent EXECUTES those engine files. T bytes fetched over the network become runn

ATM ID	Type	Status	Severity	Description
				code on the operator's host. The ONLY validation is compile(content, "<string>", "exec") at tools/plugin_update_automation.py:213 – conducted and verified as the sole compile call. That is a SYNTAX check. A syntactically valid file is exactly what a hostile payload is. There is no signature verification, NO pinned commit SHA, NO hash allowlist, NO provenance check of any kind. §11.4.252 requires a path combining dangerous capabilities to FAIL CLOSED – verify every precondition, refuse when any is unverifiable. This path combines THREE and verifies none of them. §11.4.246's supply-chain clause is the direct counterpart: dependencies are either vendored hash-verified, provenance-attested, or mirrored through a hash-pinning registry – an unattested public source is an integrity risk and here the unattested public source becomes EXECUTED CODE. THE SHARPEST FACT ABOUT THIS FINDING: the fail-closed gate produced THREE FALSE POSITIVES in this same file (:189, :215 – all triaged FALSE POSITIVE, see BOB-214's correction) while MISSING this, one genuine dangerous combination in it. This is the §11.4.201 both-directions failure – false-positive and false-negative – demonstrated inside a single file. It is the strongest available evidence that the detector classifies by local syntactic shape and never consults semantic role (the §11.4.250 primitive named in BOB-191/BOB-216). REACHABILITY, STATED HONESTLY (§11.4.6): the script is invoked by NOTHING – control-never proven (91 needle hits, 0 negative control hits) is referenced only by its own README, its docstring, and a tracker note; zero hits across *.sh / *.yaml / *.py / Makefile outside tools/. Its own README self-declares "not invoked by the normal start/stop flow". Last functional commit 2026-04-12. So this is NOT live in any automated path. It IS reachable

ATM ID	Type	Status	Severity	Description
				a hand-run --update, which is exactly what the tool exists for. Severity is Critical on the capability combination, bounded by that reachability – not on an active exploitation path. ACCEPTANCE: (1) plugin sources are pinned (commit SHA or content hash) and verified before write; (2) an unverifiable source REFUSES and names which check failed (§11.4.252) rather than proceeding on a syntax pass; (3) the compile() call is documented as a source as a syntax check that is NOT a safe gate, so nobody reads it as one; (4) a REDIRECT serving a syntactically-valid hostile payload and asserting it is REFUSED pre-fix-absent post-fix-present; (5) golden-FALSE proving legitimate pinned update still succeeds (§11.4.201(1)). DISCOVERY CHANNEL (§11.4.238): found by the concealed-hits triage stream while establishing that the gate's four hits in this area were false positives. Not by the automated QA regime and the regime that WAS pointed here reported the wrong three lines. BOB-218 Bug Queued major WHAT: tools/README.md:30 states "the backup is restored" on validation failure. Conductor-verified: shutil.copy2 appears EXACTLY ONCE in tools/plugin_update_automation.py and copies FOR only – a reverse-direction restore has ZERO occurrences (grep control-needle-proven seeing: needle 2 hits, negative control 0). The plugin write opens in mode "w", which TRUNCATES IMMEDIATELY. USER-OBSERVABLE FAILURE (the reason this is not merely a doc defect is a failure part-way through the write leaves TRUNCATED, NON-IMPORTABLE plugin on disk. operator is told the update FAILED and reasonably concludes the previous file survived – because the README says it was restored. It was not. The next ./install-plugin.sh copies the corrupt file into conductor/qBittorrent/nova3/engines/ and that search engine SILENTLY STOPS WORKING. The .bak ne

ATM ID	Type	Status	Severity	Description
				to recover DOES exist on disk; the tool ne mentions it and never uses it. This is a §11.4 documentation-layer bluff with a rea downstream consequence: the doc asserts a safety property the code does not implemen and the operator's recovery decision is ma on that false assertion. SECOND, SMALLER DIVERGENCE: the README claims JSON goes to stdout; :274 writes it to a FILE (open(args.output, "w")). THIRD, SEPARATE MINOR DEFECT in the same file (recorded he rather than as its own item because it sha the file and the fix window): _extract_ver at ~:198 uses a BAREexcept:, so a Ctrl-C during the 14-URL sweep is SWALLOWED. That a signal-handling defect, not a fail-open the gate flagged this line, but for the wr reason (it read the silent default return; real issue is the bare except catching KeyboardInterrupt). ACCEPTANCE: (1) eithe the rollback is IMPLEMENTED (restore the . on failure) or the README claim is DELETED the doc and the code must agree (§11.4.6); implemented, write to a temp file and rena atomically rather than truncating in place (2) the stdout-vs-file claim corrected; (3) the bare except narrowed so KeyboardInterrupt propagates; (4) a RED that fails the write mid-way and asserts the previous plugin is intact (post-fix) / truncated (pre-fix). REACHABILITY: same as the sibling item – h run only, not in any automated path, last functional commit 2026-04-12. Severity Maj on the harm shape, bounded by that reachability. DISCOVERY CHANNEL (§11.4.23 found by the concealed-hits triage stream. by the automated QA regime. BOB-219 Queued major WHAT, conductor-verifie docs/guides/tracker-credentials.md is TRAC Its §11.4.65-mandated export twins docs/ guides/tracker-credentials.html and .pdf b EXIST ON DISK, are NOT tracked, and ARE ignored – matched by the .gitignore

ATM ID	Type	Status	Severity	Description
				credentials deny-all at :31. The .md is rescued by an explicit allowlist entry at nobody added entries for the twins. WHY I A LIVE VIOLATION AND NOT A HYPOTHETICAL: §11.4.65 requires every in-scope Markdown document to carry synchronized .html/.pdf siblings, and §11.4.212 requires every §11.4.65-scope doc to be reachable from the README. Two artifacts that exist locally but can never be committed are, from any fresh clone, ABSENT – so every clone of this repository has a tracker-credentials guide with no exports, while the authoring host looks complete. The divergence is invisible to the machine that would notice it. WHY NOBODY SAW IT: this is a §11.4.201(6) FALSE-NULL the BOB-212 class. git status never listed the twins, so no gate and no author had a sign-off. The blind instrument and the clean tree remain the identical quiet zero. ANOTHER INSTANCE OF THE SAME MECHANISM, filed here rather than separately because it is one fix: docs/qa/BOB-124/evidence/loginctl_user_state.txt is swallowed by *_user* at .gitignore:~55 and therefore uncommitted §11.4.83 QA evidence. Its SIBLINGS IN THE SAME DIRECTORY – user1000_grepped.log, user_scope_events_head.log, user_service_lifecycle.log – ARE tracked, saved only by the robust glob negation !docs/qa/**/*.*.log. That one file is lost purely because it is .txt rather than .log. The pattern is exact: robust glob negations handle per-file and per-extension rescues leak. ACCEPTANCE: (1) both twins become trackable and are committed alongside their .md; (2) BOB-124 .txt evidence likewise; (3) whichever BOB-212 fix direction is chosen, it MUST close these – a fix that closes the future-swallow while leaving these three artifacts permanently uncommittable has addressed the mechanism and not the damage; (4) a check for a tracked .md in §11.4.65 scope has

ATM ID	Type	Status	Severity	Description
				COMMITTABLE twins, so this class cannot re silently. DISCOVERY CHANNEL (§11.4.238): found by the BOB-212 blast-radius sweep – only on its SECOND pass. The first pass filtered by a source-extension set that omitted .pdf, which hid this finding entirely. The agent re-ran with no extension filter on all 420 non-artifact paths and recorded the blind spot rather than shipping the first number. Worth keeping: an extension allowl is itself a false-null generator, which is the same shape as the defect being investigated. BOB-220 Bug Queued minor WHAT: scripts/ownership_repair.sh:891-895 runs c 755 intending to strip setuid/setgid bits. Traced live: the chmod RUNS and SUCCEEDS, the mode stays 2755. Root cause measured – chmod with a 3-DIGIT symbolic-equivalent c PRESERVES the setgid bit on DIRECTORIES; c a 4-digit form (00755) or an explicit g-s clears it. The comment at :868-884 describes strip that does not occur for directories. EFFECT: benign today – the surviving setgid bit is not itself harmful, and no defect is known to follow from it. This is filed as §11.4.6 accuracy defect: an in-source comment asserting behaviour the code does not perform. That matters because the next reader will trust it, and because a future security-relevant strip written against the same pattern would silently fail the same way. RELATION TO BOB-207: the same stream measured that after repair a new file inherits gid with THE SETGID BIT STILL SET – the two facts compound. If BOB-207 is resolved by narrowing the repair (the recommended direction), the setgid survival becomes moot for that path; the misleading comment remains. ACCEPTANCE (1) either the strip is made real (00755 c s) or the comment is corrected to state that directories retain setgid – code and comment must agree; (2) if made real, a RED assertion mode 2755 -> 0755 on a directory, since the

ATM ID	Type	Status	Severity	Description
				current form silently no-ops; (3) golden-P proving a directory that SHOULD keep its m is not gratuitously changed. DISCOVERY CHANNEL (§11.4.238): found by the BOB-207 stream while tracing an unrelated behaviou Not by the automated QA regime. BOB-22 Bug Queued critical WHAT: scripts/ pre_build_verification.sh invariant 30 (CM BASH-UNIT-TESTS-EXECUTED) enumerates suite FILESYSTEM GLOB (:1196) and calls fail() o any non-zero exit. It therefore RUNS untra suites, and it has NO concept of a test th is SUPPOSED to fail. MEASURED: tests/ pre_build/test_bob205_danger_roots_scope.s exits 1 – correctly, because it is a RED t for BOB-205, an unfixed defect. Invariant counts that as a gate failure. The predict T042 verdict is FAIL, and this is one of i three blocking causes. THE STRUCTURAL PROBLEM: §11.4.115 and §11.4.224 REQUIRE a authored FIRST and OBSERVED TO FAIL before fix exists. §11.4.135 requires the RED to persist as the permanent regression guard. the discipline mandates that failing tests exist in the tree between authoring and fi – and the blocking gate treats their exist as a defect. Following the constitution correctly makes the gate refuse the commit That is a §11.4.120 wrong-seam problem: th gate asserts "no suite fails" when the invariant it should assert is "no suite fa UNEXPECTEDLY". WHY IT SURFACED NOW: this session authored FOUR REDs across parallel streams (BOB-204, BOB-205, BOB-207, BOB-21 the first time the discipline was applied this volume. Previously REDs were flipped GREEN within the same round, so the window never spanned a gate run. The defect is no new; the exposure is. THE FORBIDDEN RESPON (§11.4.120): do NOT delete the RED to make gate green; do NOT weaken invariant 30 to ignore failures; do NOT mark the RED skip without a tracked reason. Each converts a

ATM ID	Type	Status	Severity	Description
				signal into silence. FIX DIRECTION – this §11.4.248's quarantine mechanism, which the constitution already specifies and this project has not wired: a RED declares its expected-failing status and its tracked item (a header marker, a naming convention, or a manifest), invariant 30 reads that declaration, and a declared-RED failure is reported as EXPECTED (not a fail) while an UNDECLARED failure still blocks. Crucially, a declaration must EXPIRE or be tracked – §11.4.248 pairs quarantine with a stabilisation deadline precisely so "expected to fail" cannot become permanent cover. A gate whose item closes must flip GREEN or the gate should FAIL on the stale declaration. ACCEPTANCE: (1) a declared RED does not block (2) an UNDECLARED failing suite still blocks (the §11.4.201(1) guard – a fix that ignores all failures is strictly worse than the defect); (3) a declared RED whose tracked item is CLOSED blocks, so declarations cannot remove (4) a paired §1.1 mutation removing the declaration-check makes the gate accept an undeclared failure -> gate FAILs. DISCOVER CHANNEL (§11.4.238): found by the T042 readiness preflight, before the endgame rather than during it. Not by the automated QA regime – the regime IS the thing that would have been blocked. === CORRECTED 2026-08-26 – MY FIX DIRECTION IN THIS ITEM WAS WRONG, AND THE BLOCKER IS WIDER (§11.4.6) === CORRECTION §11.4.248 IS NOT THE MECHANISM. I wrote "the §11.4.248's quarantine mechanism, which the constitution already specifies and this project has not wired." That is half right and the wrong half matters. §11.4.248(A) QUARANTINE targets FLAKY tests – its trigger is literally "two-consecutive-run-different-verdict OR rerun-after-red", and its remedy is to MOVE the suite to tests/quarantine/ and the blocking suite WITHOUT it "so green means green". §11.4.248(B) PROTECTED-SPEC is a test

ATM ID	Type	Status	Severity	Description
				plus a required-reviewer gate on MODIFYING regression test; it never says a test may fail. A RED is DETERMINISTIC and MUST STILL RUN – §11.4.115(F) and §11.4.226 require the RED verdict to be PRODUCED as the evidence. Excluding it destroys the very artifact the discipline mandates. Merging the two would also let a genuinely flaky suite hide behind a RED declaration and vice versa. What §11.4.226 legitimately supplies, and what should be INHERITED rather than re-invented (§11.4.226 is the SHAPE: a by-name list + a tracked item + a deadline + a must-only-shrink ratchet. CORRECTION 2 – THIS IS A LANGUAGE-PARITY GATE, NOT A DESIGN VACUUM. Conductor-verified: the project ALREADY runs this mechanism on the PYTHON side. <code>pytest.mark.xfail(strict=True)</code> is used deliberately for OPEN defects at <code>test_scaling/test_scaling_envelope.py:907</code> (BOB-221 documented in <code>tests/scaling/README.md:64</code>) <code>tests/stress/test_plugin_parsers_stress_chaos.py:90</code> – w/ own comments carry the ratchet discipline verbatim ("it stays visible as xfailed in every report", "then remove this xfail"). "declared expected-failure, strict so an unexpected pass fails" is ALREADY accepted policy in one language. The BASH side has an equivalent. That is the real gap. CORRECTION 3 – BOB-221 IS NOT SUFFICIENT TO UNBLOCK T A full replication of invariant 30 measures RAN=38 FAILED=4 SKIPPED=2: <code>rc=124 300s</code> <code>test_compute_badges_carrier_match.sh</code> <- a HANG, not a RED. Blocks independently. File separately. <code>rc=1 3s</code> <code>test_ownership_gid_agreement.sh</code> <- BOB-207's RED (a SECOND in-glob RED; this assumed one) <code>rc=1 1s</code> <code>test_bob205_danger_roots_scope.sh</code> <- BOB-205's RED <code>rc=127 9s</code> <code>test_check_cm_lan_routes_authenticated.sh</code> CONTAMINATED: a sibling stream was writing <code>lan_route_auth_analyzer.py</code> during and after

ATM ID	Type	Status	Severity	Description
				the run (§11.4.119 – the surveying stream not own that resource and correctly declining to adjudicate). rc=127 is the §11.4.1 scri internal FAIL-bluff signature, not a product verdict either way. Returned to its owning stream. ALSO: BOB-204's and BOB-212's REDS share tests/security/ test_gitignore_swallow_is_loud.sh, which is OUTSIDE invariant 30's glob and executed by NOTHING (BOB-222) – they do not block today and widening the glob would immediately add another blocker. THE DESIGN, RECOMMENDED: two-place declaration + SSoT expiry + string semantics. THREE independent facts must agree before a failure is excused – (1) the suite listed in scripts/lib/expected_red.sh, a fact the PRODUCING streams do not own; (2) the suite file itself carries# EXPECTED-RED: with the SAME id; (3) <ID> is OPEN in docs workable_items.db, FAIL-CLOSED on unreadable store or unknown id (§11.4.252). Plus STRIP XFAIL PARITY: a declared suite that PASSES blocks, which mechanically forces the ratio to shrink in the same commit as the fix. EXIT-CODE CONVENTION REJECTED ON MEASURED EVIDENCE – and I had flagged it as promising. The non-{0,1} exit space is ALREADY CLAIMED THIS REPO FOR THE OPPOSITE MEANING: test_gitignore_swallow_is_loud.sh uses exit = "INSTRUMENT BROKEN or SECRET LEAK – verdict void", and test_bob205_danger_roots_scope. uses exit 3 = ABORT (instrument blind). Both were chosen precisely so a blind harness could never be read as a verdict. Overloading either for "expected red" would make a genuinely BLIND INSTRUMENT INDISTINGUISHABLE FROM A DECLARED RED – the gate would swallow exactly the failure class those codes exist to make loud. It also fails acceptance (4) (an intent carries no item id) and (3) (nothing to recheck), and it puts the declaration inside PRODUCER, collapsing producer!=gate (§11.4.240/§11.4.249). HONEST BOUNDARY ON

ATM ID	Type	Status	Severity	Description
				DESIGN (§11.4.240): it achieves separation LOCATION and MULTI-PLACE AGREEMENT, not by true CAPABILITY, which §11.4.240 prefers. There is no PR/CODEOWNERS seam here to enforce capability – CLAUDE.md Hard Stop #1 removes all CI/PR gates, so the CODEOWNERS half of §11.4.248(B) is STRUCTURALLY UNENFORCEABLE on this project. What the design buys is that silencing a failure requires a coherent three place story visible in a diff, instead of a comment line. THE RED: tests/pre_build/test_bob221_expected_red_declaration.sh. I extract invariant 30's loop and its block predicate VERBATIM FROM THE LIVE GATE BY CONTENT MARKER (never line number) and RUN them against a hermetic scratch PROJECT_ROOT so the logic under test is the gate's own text, not a paraphrase (§11.4.226 anti-echo). Measured RED: 3 of 5 properties unmet, existing P3 is asserted on the blocking REASON, not merely on "it blocked" – today everything blocks, so a bare did-it-block check would pass FOR THE WRONG REASON and keep passing the anti-rot half were never built. Polarity flip proven with the BYTE-IDENTICAL test against a sandbox carrying the patch: exit all five properties met. Four paired §1.1 mutations each kill exactly ONE property, to-one, no cross-talk. THE PATCH: 72 line hunks + one new file scripts/lib/expected_red.sh,git apply –checkclean, NOT APPLIED. Operational note the landing commit must not miss: the table names test_bob205_danger_roots_scope.sh and test_ownership_gid_agreement.sh, but NEITHER carries an# EXPECTED-RED: marker today (verified 0 in both) — without those two one-additions the gate blocks them with"declared the file carries no matching marker", which is the fail-closed design working as intended, no bug. SEQUENCING RECOMMENDATION: (iii), bounded variant of (i) — land the mechanism declaring exactly BOB-205, BOB-207 and this

ATM ID	Type	Status	Severity	Description
				item's own suite, add the two missing markers to the rc=124 hang as its own item, return rc=127 to its owner, THEN run T042. THE HONEST COST, STATED NOT HIDDEN: this modifies the blocking gate BEFORE that gate ever run clean, so the mechanism's first full-gate exercise IS the T042 run. The circularity is unavoidable and IS this item (§11.4.120 — a gate whose precondition is produced by the work of the gates): a clean T042 run is impossible while a RED exists, so “validate the gate before changing it” cannot be satisfied. Risk reduced as far as possible without running the gate (proven-applicable patch, proven polarity flip, four bit mutations, P2 undeclared-failures-still-block asserted and mutation-proven) but NOT eliminated. Against (ii) fix-the-four-defects-first all four are Queued and genuinely deferred; BOB-212's fix direction is an explicitly unresolved §11.4.66 OPERATOR decision whose own test is deliberately written fix-direction-agnostic so not to prejudge it — choosing (ii) means making that decision on the operator's behalf, plus four implementations and four reviews, and it STILL leaves the hang and the rc=127 blocking. RECURSION NOTE: this item's own RED is in-progress and failing, so it is itself a 5th blocker until the mechanism lands — at which point its already-present marker and table row clear it. The mechanism is its own first customer. WHAT: scripts/pre_build_verification.sh:1196 enumerates bash suites with a glob covering EXACTLY three directories: “\${PROJECT_ROOT}/tests/unit/test_*.sh · “\${PROJECT_ROOT}/tests/pre_build/test_*.sh · “\${PROJECT_ROOT}/tests/hooks/test_*.sh tests/security/ is NOT among them. Conductor-verified verbatim. So tests/security/test_gitignore_swallow_is_loud.sh — a RED authored this session with a golden-FALSE guard proving real secrets stay ignored — is executed by NOTHING. It is a guard that guards nothing and its silence is indistinguishable from success.
BOB-222	Bug	Queued	major	

ATM ID	Type	Status	Severity	Description
				THIS IS THE THIRD OCCURRENCE OF ONE CLASS. The driver's OWN comment at :1030 records the history: the same defect class "existed for tests/pre_build/test_*.sh" and was fixed by adding that directory to the glob. It then recurred one directory over (tests/hooks), fixed the same way. Now tests/security. Each fix was a new glob entry; none addressed why a new test directory is invisible by default. §11.4.250 APPLIES DIRECTLY: three symptoms, one primitive. The primitive is a hand-maintained enumeration that must be edited in lockstep with the tree, whose failure mode is SILENT. It is the identical shape as BOB-205's DANGER_ROOTS and BOB-212's .gitignore allowlist — three separate hand-maintained lists in this repo, all three measured to have silently missed something. Adding tests/security to the glob would be the fourth instance of layer N+1. NO THE GATE ALREADY HAS THE RIGHT INSTINCT at :1220: it fails when the glob matches NOTHING ("the glob is blind"). That guard catches a TOTALLY blind glob but not a PARTIALLY blind one — the more common and more dangerous case, because a partially blind glob still reports a healthy count. FIX DIRECTION (§11.4.251 role of data-pack): derive the suite set from the tree of every tests/**/test_*.sh — minus a DECLARED §11.4.224(E) exclusion fence with a justification per entry. Then a new test directory cannot be invisible, and a deliberate exclusion is visible and reasoned. If a hand list is kept, the omission guard is the same computation, so the derivation must be built either way. ACCEPTANCE: (1) test security suites execute; (2) a NEWLY CREATED tests//test_x.sh is picked up with no hand editing that is the invariant that stops the recurrence and without it this is just the fourth patch; (3) RED creating such a directory and asserting it runs; (4) the :1220 blind-glob guard is extended to catch PARTIAL blindness, not only total. DISCOVERY CHANNEL (§11.4.238): found by the T042 readiness preflight. Not by the automated

ATM ID	Type	Status	Severity	Description
BOB-223	Bug	Queued	major	QA regime — and notably not by the two previous fixes of this same class, neither of which asked why it happened. WHAT, both conductor-verified with control needles: CM-SCRIPT-DOCS-SYNC occurrences in scripts/pre_build_verification.sh = 0 (control needle CM-MARKDOWN-EXPORT-SYNC = 6, so instrument sees). §11.4.44 revision-header enforcement likewise 0 hits (control needle 3). Neither §11.4.18's companion-doc mandate nor §11.4.44's revision header is enforced by any invariant. THE PERVERSE CONSEQUENCE, MEASURED: a new script with NO companion doc passes the gate. Writing the companion doc that §11.4.18 MANDATES creates docs/scripts/.md, which invariant 16 (CM-MARKDOWN-EXPORT-SYNC) then requires to have .html and .pdf twins — and their absence is a BLOCKING failure. This session hit it exactly in docs/scripts/test_gitignore_swallow_is_loud. {html,pdf} are missing and are two of invariant 16's six violations. So the gate's incentive gradient points AWAY from the constitution: complying with §11.4.18 is punished, ignoring it is free. That is worse than an ungated rule — a rule the machinery actively discourages. §11.4.227 IS THE GOVERNING ANCHOR: a naive gate that exists only in prose is gate debt, and a ledger of unimplemented CM-* names must be monotone-decreasing. CM-SCRIPT-DOCS-SYNC named in §11.4.18's own text and implemented nowhere — one concrete row of exactly the 5 unimplemented population §11.4.227 was meant to shrink. ACCEPTANCE: (1) either CM-SCRIPT-DOCS-SYNC is implemented, or it is registered as a deferral pointing at a tracked §11.4.197 item; silent absence is what §11.4.227 forbids; (2) so for §11.4.44's header check; (3) whichever is chosen, the incentive inversion is closed: it must never be cheaper to skip a mandated doc than to write one — if invariant 16 will demand twin docs, the doc-creation path must produce them, or

ATM ID	Type	Status	Severity	Description
				invariant 16 must scope out docs/scripts until it can; (4) a paired §1.1 mutation per gate implemented. HONEST NOTE ON SCOPE: this item does not argue §11.4.18 should be enforced immediately — that is an operator call about debt priority (§11.4.66). It argues the CURRENT state is incoherent: an unenforced mandate whose observance triggers a different gate's failure. Either enforce both ends or neither. DISCOVERY CHANNEL (§11.4.238): found by the T042 readiness preflight while explaining why a new file broke invariant 16. Not by the automated QA regime. WHAT: a replication of invariant 30 (same global skip list, same BOBA_PREBUILD_NESTED same timeout 300) measured RAN=38 FAILED SKIPPED=2. One of the four failures is tests/unit/test_compute_badges_carrier_match.sh exiting rc=124 after the FULL 300 seconds — it does fail, it HANGS. WHY IT IS FILED SEPARATELY AND URGENTLY: it blocks invariant 30, and therefore T042, INDEPENDENTLY of BOB-221. The assumption in BOB-221 that landing an expected-RED mechanism unblocks T042 is FALSE as stated — two of the four failures are REDs, one is this hang, one is contaminated. Landing the mechanism alone leaves T042 blocked by this suite. IT MUST NOT BE SILENT BY A DECLARATION. A hang is a §11.4.232(C) liveness failure, not a verdict: a wedged op and progressing op both look like “not finished yet” and marking it expected-to-fail would convert §11.4.201(6) false-null into permanent coverage. This is exactly the abuse the expected-RED design property (2) exists to prevent, so it must be triaged as its own defect. INVESTIGATION DIRECTION (§11.4.102 first, no guessing): rc=124 is the timeout(1) signature. Determine WHERE the wedges — a consumer blocked on an unclosed producer write-end is the documented shape of this here (§11.4.201(12) records that a background watchdog spawned inside a \$(...) command-
BOB-224	Bug	Queued	critical	

ATM ID	Type	Status	Severity	Description
				substitution inherits the pipe write-end, and early disarm orphans its sleep grandchild, so substitution stalls for the FULL budget while every verdict and exit code stays CORRECT). 7 signature — full budget, correct verdicts — matches rc=124 exactly and should be the FILE hypothesis tested, not the last. The countermeasure is documented: redirect the watchdog subshell fds away from the cmd-sub pipe, or have the probe write to a file. Do NOT assume that is the cause; it is the highest-priority hypothesis given the recorded precedent. ACCEPTANCE: (1) the wedge point is identified with captured evidence, not inferred; (2) the suite completes deterministically well inside budget; (3) a RED reproducing the hang, so a regression cannot silently re-wedge (a timeout-based assertion, since the failure IS the duration); (4) NOT closed by a declaration or raising the timeout — raising the budget hides FILE DATE: the suite is dated 2026-08-21, so the hang predates this session. DISCOVERY CHANNEL (§11.4.238): found by the BOB-221 design stream while surveying invariant 30 more widely than its brief required — the T042 preflight had verified only 3 of 38 suites and stated that boundary honestly, which is what prompted the wider survey. WHAT, conductor-verified: .gitignore:266 ignored *.docx globally. The §11.4.65 export pipeline GENERATES .docx twins — the workable-item export produces Issues.docx / Fixed.docx / Issues_Summary.docx / Fixed_Summary.docx and the doc exporter produced docs/scripts/check_cm_lan_routes_authenticated.docx during round 9. Every one of them is untrackable by construction: they exist on disk, git will never notice them, and no gate can notice because the absence is silent. WHY THIS IS THE BOB-212 CLASS, NOT A DUPLICATE OF IT: BOB-212 is a deny-all glob plus a hand-maintained per-file ALLOWLIST, where new files leak through the
BOB-225	Bug	Queued	major	

ATM ID	Type	Status	Severity	Description
				gaps in the list. This is a deny-all glob with NO allowlist at all for a file type the project MANDATES producing. The mechanism differs: the false-null is identical — git status stays silent so the exporter appears to succeed and the artifact appears to exist. It is filed separately §11.4.214 (distinct-but-similar), with BOB-212 BOB-219 as siblings. THE TENSION TO RESOLVE HONESTLY: §11.4.153 mandates a FOUR-format export (HTML + PDF + DOCX) for its document class, and §11.4.65 governs the twins generally. So the constitution requires producing an artifact the repository is configured to refuse. One of the two is wrong and the resolution is an operator decision (§11.4.66): (a) the .docx mandate appears here and the glob must carve out generated artifacts; twins; (b) .docx is deliberately untracked as a heavy binary derivative regenerable per §11.4.6 from its .md, in which case the EXPORTERS should stop producing it, or produce it into an explicitly untracked location, and the §11.4.153 four-format requirement should be recorded as consciously not-adopted rather than silently unmet. What is NOT acceptable is the present state: generate it, ignore it, and let both the mandate and the glob appear satisfied. MEASURED SCOPE (CORRECTED — my first version of this item said ‘tracked .docx files = 0’, which was FALSE; recorded here per §11.4.6 rather than silently amended): 2 .docx files ARE tracked and 1140 exist repo-wide, so 1138 are untracked-and-ignored. (A second correction, also recorded here rather than amended: my first correction said 344, which counted only the docs/ subtree. Two numeric errors in a row on one item — the pattern is that each was measured with a narrower instrument than the claim it supported, which is the §11.4.201(7)(c) path-in-part-of-the-instrument failure applied to my reporting.) AND THE TWO SURVIVORS ARE THE INTERESTING PART: they are docs/features/Status.docx and docs/features/Status_Summary.docx — precisely the pair

ATM ID	Type	Status	Severity	Description
				§11.4.153 explicitly mandates as a FOUR-form export. So the only two DOCX artifacts git can have are the two a specific anchor named, and the two survive not by rule but because they are already in the index. That makes this a PARTIAL condition, not a total one, and the two survivors are the interesting part — they are in the same position as frontend/e2e/credentials.spec.ts under BOB-212: alive only because they are already in the index, since git honours the index over .gitignore for tracked paths. One delete-re-add, one file move, and they vanish silently like any other. So the state is worse than a clean 'we never track docx': it is an inconsistent one where two artifacts appear to prove the form IS tracked while 342 prove it is not. ACCEPTANCE: (1) the operator decision above taken and recorded; (2) whichever way, the exporter and the ignore rule AGREE — if .docx not tracked, nothing should silently generate into a tracked doc directory; (3) if carved out, check that a generated twin is actually tracked so this cannot recur silently; (4) §11.4.153 compliance is either met or recorded as an honest gap, never left implicitly failing. DISCOVERY CHANNEL (§11.4.238): found by the BOB-102 round-9 remediation stream when it regenerated its own guide twins and noticed the .docx could not be added. Not by the automated QA regime — and the regime cannot see it, which is the point. WHAT: ownership_repair walks a declared root and repairs items whose uid is not the operator. The unit suite seeds a foreign uid only onto FILES and SYMLINKS; interior DIRECTORIES stay operator-owned (case 22 seeds only a foreign uid declared ROOT, on the failure path). Production first-start repairs exactly the untested shape. MANIFEST: tests/unit/test_ownership_repair:seed_tree/seed_wrong; scripts/ownership_repair.sh walk at :994. REPRO: seed tree whose interior directories carry uid 1000
BOB-226	Task	Queued	major	

ATM ID	Type	Status	Severity	Description
				via podman unshare, run the repair, observe automated assertion covers the outcome. WHAT: UNTESTED: the unprivileged harness cannot create symlinks inside a directory it no longer owns, which cases 8/9/18 require. The DECLARATION GAPS note cross-references tests/ownership/test_container_writes_owned_files.py, but that test covers the CREATION side (FR-002), not the repair-side walk. ACCEPTANCE: an integration layer test (where the unprivileged-harness constraint does not bind) that seeds foreign-owned interior directories, runs the repair, and asserts post-state ownership plus mode preservation. Surfaced by the BOB-207 independent review 2026-08-27. WHAT: three of the four artifacts of the CM-LAN-ROUTES-AUTHENTICATED pre-build gate are untracked in git. MEASURED 2026-08-27 with ls-files -error-unmatch, control-needed against known-tracked file: scripts/pre_build/lan_route_auth_analyzer.py UNTRACKED, scripts/pre_build/check_cm_lan_routes_authenticated.py UNTRACKED, tests/pre_build/test_check_cm_lan_routes_authenticated.sh UNTRACKED; only docs/scripts/check_cm_lan_routes_authenticated.md is TRACKED. IMPACT: (1) losing this checkout loses an entire security gate plus 197 assertions; (2) round-over-round diff claim across review rounds 8 through 14 was ever checkable, because no committed baseline exists - the same §11.4 evidence-custody failure the BOB-195 chain handles independently; (3) a fresh clone runs a pre-build gate whose implementation is absent. ACCEPTANCE: all four artifacts tracked and committed, and a gate asserting that every executable a pre-build invariant invokes is tracked. Surfaced by the BOB-102 round-13 remediation and independently verified 2026-08-27.
BOB-227	Bug	Queued	critical	
BOB-228	Task	Queued	minor	WHAT: §11.4.212 makes the main README the canonical entry point for ALL project

ATM ID	Type	Status	Severity	Description
				documentation, with no §11.4.65-scope doc reachable by no link path. docs/scripts/check_cm_lan_routes_authenticated.md is not reachable from README.md. MEASURED 2026-08-27: grep -c for the guide name in README.md returns 0, control-neededled again. doc README does link (CONTINUATION returns 1), so the instrument is not blind. ACCEPTANCE README links the guide directly or transitively and the doc-link generator covers docs/scripts/ the next such guide cannot land orphaned. Surfaced by the BOB-102 round-13 remediation and independently verified 2026-08-27.